МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра промышленной информатики

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой_____В.А. Холопов

«_»____июня_____ 2024 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки бакалавров 09.03.01 Информатика и вычислительная техника

На тему: «Информационно-управляющая система интерактивного взаимодействия со стендовым оборудованием».

Обучающийся _____ *Иванисенко Захар Ильич*
подпись *Фамилия, имя, отчество*

шифр 20И0419
группа ИВБО-07-20

Руководитель работы _____ *к.т.н., доцент* *Курнасов Е.В.*
подпись

Консультант _____ *старший преподаватель* *Антонов С.В.*
подпись

Консультант _____ *к.э.н., доцент* *Казанская И.В.*
подпись

Нормоконтроль _____ *старший преподаватель* *Володина А.М.*
подпись

Москва 2024 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра промышленной информатики

СОГЛАСОВАНО	УТВЕРЖДАЮ
Заведующий кафедрой _____ *подпись*	Директор института _____ *подпись*
Холопов Владимир Анатольевич	Зуев Андрей Сергеевич
« » апреля 2024 г.	« » апреля 2024 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

Обучающийся Иванисенко Захар Ильич
Фамилия Имя Отчество

Шифр 20И0419

Направление подготовки 09.03.01 Информатика и вычислительная техника
индекс направления *наименование направления*

Группа ИВБО-07-20

1. Тема выпускной квалификационной работы: «Информационно-управляющая система интерактивного взаимодействия со стендовым оборудованием».

2.Цель и задачи выпускной квалификационной работы

Цель работы: повышение уровня интерактивности демонстрационного функционала лабораторного стенда при проведении выставочный мероприятий.

Задачи работы: проанализировать предметную область, выполнить структурно-функциональный анализ в нотациях IDEF0 и BPMN, разработать архитектуру ИУС, разработать логическую модель данных, реализовать необходимые программные компоненты ИУС.

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1 1.1 1.2 1.3 1.4 1.5	Аналитический раздел Обзор программно-технических решений по интерактивному взаимодействию с техническими устройствами Описание методов обмена данными с промышленными устройствами Описание используемого в ВКР стендового оборудования Определение требований к разрабатываемой ИУС Постановка цели и задач ВКР		06.05.2024
2 2.1 2.2 2.3	Проектный раздел Моделирование элементов ИУС Разработка архитектуры ИУС Разработка логической модели данных ИУС		16.05.2024
3 3.1 3.2 3.3 3.4	Программный раздел Выбор платформ и средств разработки Реализация базы данных Разработка стендового компонента системы Реализация серверного компонента системы		23.05.2024
4 4.1 4.2	Экономический раздел Организация и планирование работ по теме Расчет затрат на проведение работ по теме		24.05.2024
5	Аннотация, введение, заключение, список источников		26.05.2024
6	Презентация		27.05.2024
7	Нормоконтроль		28.05.2024

4. Перечень разрабатываемых документов и графических материалов

Пояснительная записка, презентация в PowerPoint.

5. Руководитель и консультанты выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	доцент	Курнасов Евгений Вячеславович	
Консультант специальной части ВКР	старший преподаватель	Антонов Сергей Валерьевич	
Консультант по экономической части ВКР	доцент	Казанская Ирина Витальевна	

Задание выдал
Руководитель
ВКР:_____
подпись
«___» апреля 2024 г.

Задание принял к исполнению

Обучающийся:_____
подпись
«___» апреля 2024 г.

АННОТАЦИЯ

Тема выпускной квалификационной работы: «Информационно-управляющая система интерактивного взаимодействия со стендовым оборудованием».

Выпускная квалификационная работа включает в себя: введение, 4 раздела, заключение, список использованных источников, а также шесть приложений.

В Аналитическом разделе рассмотрены виды технологий взаимодействия с промышленным оборудованием, существующие проблемы в этой области и некоторые решения, предлагаемые крупными компаниями. Также в этом разделе описаны требования к разрабатываемой системе. Раздел содержит 4 рисунка, 5 таблиц и 13 страниц

В Проектном разделе составлены модели системы в нотациях IDEF0 и DFD, разработана архитектура системы и составлена модель данных системы с учетом ранее выявленных требований. Раздел содержит рисунков, таблиц и 10 страниц

В Программном разделе выбраны используемые средства разработки с учетом ранее выявленных требований. Также в данном разделе реализована база данных, реализован серверный компонент системы и разработан стендовый компонент системы. Раздел содержит 7 рисунков и 21 страниц.

Экономический раздел включает в себя планирование работ по ВКР, график выполнения работ и стоимость их проведения. Раздел содержит 2 рисунка, 7 таблиц и 11 страниц.

Выпускная квалификационная работа состоит из 57 страниц.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 8
1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ 9
1.1 Обзор программно-технических решений по интерактивному взаимодействию с техническими устройствами 9
1.1.1 Анализ технологии интерактивного взаимодействия с техническим оборудованием 9
1.1.2 Программно-технические решения от компании Яндекс 10
1.1.3 Программно-технические решения от других компаний 12
1.1.4 Система предоставляющие доступ к программно-техническому оборудованию 15
1.1.5 Недостатки существующих систем 16
1.1.6 Технологии, реализующие поставленные задачи 17
1.2 Описание методов обмена данными с промышленными устройствами 18
1.2.1 TCP/IP модель в производственных стендах 18
1.2.2 Семейство HTTP протоколов 19
1.3 Описание используемого в ВКР стендового оборудования 19
1.4 Определение требований к разрабатываемой ИУС 20
1.5 Постановка цели и задач ВКР 22
2 ПРОЕКТНЫЙ РАЗДЕЛ 23
2.1 Моделирование элементов ИУС 23
2.1.1 Концептуальная модель системы 23
2.1.2 Динамическая модель в нотации BPMN 26
2.2 Разработка архитектуры ИУС 27
2.2.1 Диаграмма компонентов 27
2.2.2 Общая архитектурная схема 28
2.3 Разработка логической модели данных ИУС 30
3 ПРОГРАММНЫЙ РАЗДЕЛ 33
3.1 Выбор платформ и средств разработки 33

3.1.1 Выбор модели ПЛК для стендового оборудования 33
3.1.2 Выбор средств разработки для стендового оборудования 33
3.1.3 Выбор ПК для реализации серверного компонента 33
3.1.4 Выбор операционной системы для реализации серверного компонента 34
3.1.5 Выбор средств разработки отдельных компонентов серверной части .. 35
3.1.6 Выбор СУБД для системы 36
3.2 Реализация базы данных 36
3.2.1 Создание сервера базы данных 36
3.2.2 Создание базы данных на сервере 38
3.3 Разработка стендового компонента системы 40
3.3.1 Создание TCP сервера для сетевого взаимодействия с системой 40
3.3.2 Написание простого сценария на стендовом оборудовании 41
3.4 Реализация серверного компонента системы 42
3.4.1 Создание Telegram бота 42
3.4.2 Создание TCP клиента для сетевого взаимодействия с системой 45
3.4.3 Создание панели конфигурации 47
3.4.4 Интеграция голосового помощника в систему 48
3.4.5 Создание дашборда для интерактивного представления информации 49
3.4.6 Сборка и развертка проекта 52
4 ЭКОНОМИЧЕСКИЙ РАЗДЕЛ 55
4.1 Организация и планирование работ по теме 55
4.2 Расчет затрат на проведение работ по теме 60
ЗАКЛЮЧЕНИЕ 66
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 67
ПРИЛОЖЕНИЯ 69
Приложение А 70
Приложение Б 72
Приложение В 74
Приложение Г 77
Приложение Д 81

Приложение Е..... 83

ВВЕДЕНИЕ

В современном мире, где технология играет ключевую роль в производственных процессах, возникает необходимость в разработке инновационных подходов к управлению и взаимодействию с техническим оборудованием. Все актуальнее становится вопрос о создании такой системы, которая позволит инженерам и другим пользователям удобно работать с оборудованием в удаленном формате. Необходимо сделать абстрактную реализацию программного комплекса, не привязанную к конкретному стенду и привести пример ее работы, используя существующий физический объект.

Актуальность проблемы для области инженерии и промышленного дизайна состоит в том, что стендовое оборудование является неотъемлемым инструментом для исследования, разработки и тестирования. Одной из существенных проблем современной промышленности является необходимость предоставления удаленного интерактивного доступа к такому оборудованию, что позволяет инженерам вносить изменения, проводить тестирование и анализировать данные в реальном времени, не завися от своего физического расположения.

Выпускная квалификационная работа посвящена разработке информационно-управляющей системы интерактивного взаимодействия со стендовым оборудованием. Объектом исследования системы является стендовое оборудование, а объектом автоматизации – механизм интерактивного взаимодействия со стендовым оборудованием. Концепция системы основана на использовании программно-технических средств для управления и мониторинга стендов, что включает в себя элементы дистанционного управления и интерфейсы для взаимодействия с пользователями. Особенностью предлагаемой системы является ее универсальность и способность адаптации к различным типам стендов, таким как вагонетка, что делает ее востребованной среди инженеров и специалистов, работающих в областях, где требуется высокий уровень гибкости и масштабируемости решений.

1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

1.1 Обзор программно-технических решений по интерактивному взаимодействию с техническими устройствами

1.1.1 Анализ технологии интерактивного взаимодействия с техническим оборудованием

Стоит уделить внимание такому понятию как интерактивность при взаимодействии со стендовым оборудованием.

Интерактивность в контексте данного примера означает, что система предоставляет пользователю функционал, который позволяет ему активно и адекватно реагировать на действия пользователя, делая систему «умной» и способной обрабатывать информацию, полученную от пользователя, и на её основе вырабатывать конкретные решения. Это свойство характеризует систему как способную к взаимодействию с пользователем, позволяя ему участвовать в управлении процессом и предоставляя возможность для постоянного двустороннего общения и эффективных коммуникаций. Интерактивность также может быть показателем интеллектуальности системы, позволяя ей адаптироваться к действиям пользователя и предоставлять соответствующую обратную связь или конструктивную поддержку.

Рассмотрим различные примеры интерактивного интерфейса при работе с техническим оборудованием:

- управление оборудованием со смартфона: Этот метод позволяет пользователям управлять техническим оборудованием через мобильные приложения, которые могут быть разработаны для управления различными устройствами, такими как освещение, термостаты, и даже автомобили. Приложения могут использовать различные технологии, включая Bluetooth, Wi-Fi, и даже NFC, для связи с оборудованием. Пользователи могут отправлять команды, например, «включить свет» или «установить влажность на заданный диапазон», используя голосовые команды или жесты на экране смартфона;

- управление оборудованием с помощью голосового ассистента: Голосовые ассистенты позволяют пользователям управлять техническим оборудованием с помощью голосовых команд. Это может включать в себя управление освещением, термостатами, и даже умными домашними устройствами, такими как умные колонки и телевизоры. Голосовые ассистенты используют технологии распознавания речи и машинного обучения для понимания команд пользователя и выполнения соответствующих действий;
- управление оборудованием через панель виджетов или дашборд: Панели виджетов или дашборды предоставляют визуальный интерфейс для управления техническим оборудованием. Пользователи могут настраивать виджеты или элементы дашборда для отображения информации о состоянии устройств, таких как температура, уровень заряда батареи, и даже использование энергии. Это позволяет пользователям быстро получать обзор состояния своих устройств и принимать решения на основе этой информации;
- управление оборудованием с использованием приложений виртуальной и дополненной реальности: Приложения виртуальной и дополненной реальности (VR и AR) предоставляют новые способы взаимодействия с техническим оборудованием. Например, пользователи могут использовать VR-очки для виртуального обзора своего дома и управления умными устройствами, такими как освещение и термостаты, в виртуальной среде. AR-приложения могут позволять пользователям взаимодействовать с физическим оборудованием через виртуальные интерфейсы, например, для настройки освещения или управления умными замками.

Каждый из этих методов предлагает уникальные преимущества и может быть выбран в зависимости от предпочтений пользователя, типа оборудования и контекста использования.

1.1.2 Программно-технические решения от компании Яндекс

Умный дом с Алисой от Яндекса представляет собой интегрированную систему, которая позволяет управлять различными устройствами в доме с

помощью голосовых команд. Это достигается благодаря интеграции с сервисом Яндекс.Алиса, который использует искусственный интеллект для понимания и выполнения команд пользователя. Внешний вид Яндекс станции представлен на Рисунке 1.1.

Рисунок 1.1 — Умная станция «Алиса» от Яндекса

Интерактивность умного дома с Алисой обеспечивается через голосовые команды, которые пользователь может отправлять через различные устройства, такие как смартфоны, планшеты, телевизоры и даже через голосовые ассистенты, такие как Яндекс. «Алиса». Когда пользователь говорит команду, Алиса обрабатывает ее, определяет, какое действие нужно выполнить, и отправляет соответствующие команды на устройства в доме.

Умный дом с Алисой может управлять широким спектром устройств, включая освещение, кондиционеры, термостаты, дверные замки, камеры видеонаблюдения и многое другое. Это позволяет пользователям автоматизировать и упростить управление своим домом, делая его более удобным и эффективным.

Для работы умного дома с Алисой необходимо подключить к системе устройства, которые поддерживают интеграцию с Яндекс. «Алисой». Это могут быть как умные устройства, так и традиционные устройства, которые могут быть адаптированы для работы с Алисой через специальные адаптеры или интеграции.

1.1.3 Программно-технические решения от других компаний

Рассмотрим схожие решения, которые позволят нам еще лучше понять потребности пользователей и решения, удовлетворяющие эти предложения.

Компания Wirenboard, специализирующаяся на производстве умных устройств, предоставляет удобный веб-интерфейс удаленного взаимодействия с ними.

Данное решение, использующее технологии интернета вещей, помогает во взаимодействии с умными домами, управление которыми производится посредством интуитивно понятного интерфейса. Также система предоставляет удобные дашборды для анализа текущей обстановки в помещении.

Система Wirenboard работает на основе модульной архитектуры, что позволяет легко интегрировать различные умные устройства, такие как сенсоры температуры, освещения, датчики движения и другие. Пользователи могут настраивать и управлять этими устройствами через единый веб-интерфейс, что значительно упрощает процесс управления умным домом.

В качестве интерактивной платформы служит система для анализа текущей обстановки в помещении система предоставляет детальные дашборды, на которых отображаются данные с различных сенсоров. Это позволяет пользователям быстро оценить состояние помещения и принимать решения на основе актуальной информации. Например, на дашборде можно видеть, какие комнаты в доме сейчас освещены, какая температура в разных помещениях, и так далее. Ниже, на Рисунке 1.2 представлен внешний вид дашбордов от Wirenboard.

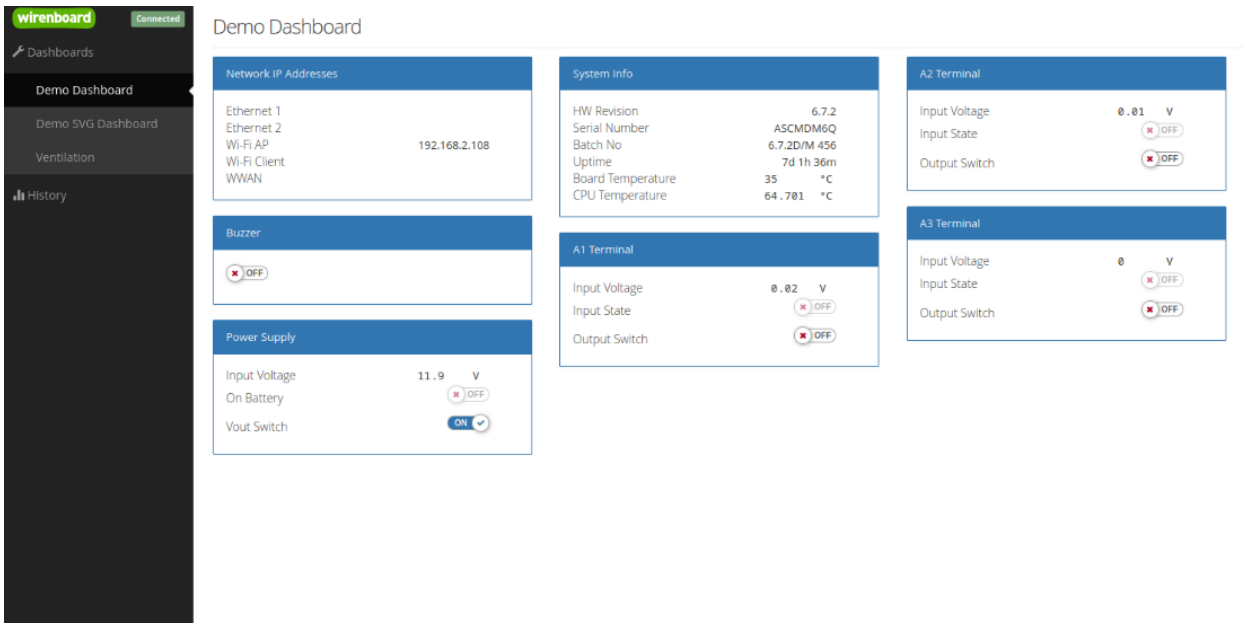

Рисунок 1.2 — Интерфейс Wirenboard

Гибкость системы состоит в том, что Wirenboard поддерживает интеграцию с другими умными системами и сервисами, что позволяет создавать более сложные и интегрированные сценарии управления. Например, можно настроить систему так, чтобы при определенном уровне освещенности в комнате автоматически включалось освещение в соседней комнате.

Также одним из ключевых аспектов гибкости системы Wirenboard является возможность создания персонализированных сценариев управления. Например, пользователи могут настроить систему так, чтобы при определенном времени дня освещение в гостиной автоматически включалось, а при выходе из дома – выключалось. Это делает управление умным домом более удобным и эффективным.

В целом, система Wirenboard предлагает мощные и гибкие инструменты для управления умным домом, делая процесс управления более удобным и эффективным.

Умный дом от Xiaomi представляет собой комплексное решение для автоматизации и управления различными устройствами в доме с помощью смартфона или компьютера. Это включает в себя управление освещением, температурой, музыкой, безопасностью и другими функциями. Система работает на базе интернета вещей, что позволяет устройствам в доме общаться между собой и с пользователем через приложение Mi Home.

Гибкость умного дома Xiaomi обусловлена возможностью интеграции с множеством устройств от различных производителей, а также поддержкой различных протоколов связи. Пользователи могут настроить сценарии для автоматического управления устройствами в зависимости от времени суток, наличия людей в доме или других условий. Это позволяет создавать персонализированные сценарии для улучшения комфорта и эффективности использования ресурсов.

Интерактивность умного дома Xiaomi достигается за счет использования голосовых команд и управления через приложение Mi Home. Пользователи могут управлять устройствами голосовыми командами, например, «Включить свет в спальне» или «Установить температуру в гостиной на 22 градуса». Приложение также предоставляет визуальные инструменты для настройки и мониторинга состояния устройств в доме.

Внешний вид интерфейса представлен на Рисунке 1.3.

В целом, умный дом от Xiaomi предлагает удобный и инновационный способ управления различными аспектами домашней жизни, делая его более удобным и эффективным.

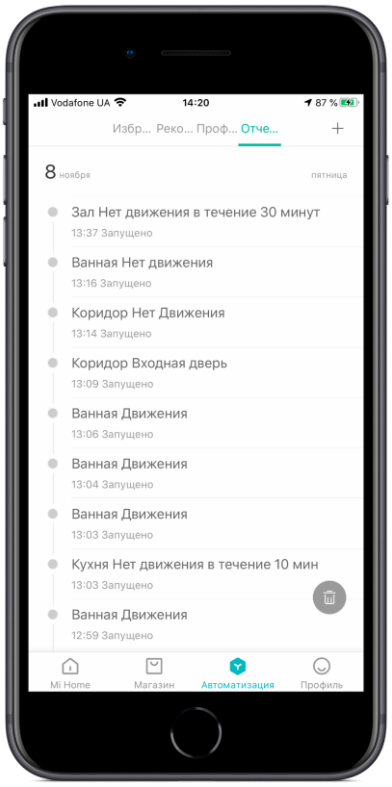

Рисунок 1.3 — Интерфейс мобильного приложения Xiaomi для управления умным домом

1.1.4 Система предоставляющие доступ к программно-техническому оборудованию

Система, предоставляющая доступ к программно-техническому оборудованию, обычно относится к комплексному программному обеспечению, которое позволяет управлять и мониторить различные аспекты технического оборудования, такие как серверы, сетевое оборудование, системы хранения данных и другие устройства. Эти системы могут быть как встроенными в оборудование, так и работать как отдельное программное обеспечение, установленное на компьютере или сервере.

Система обычно требует аутентификации пользователя для доступа к управляемым устройствам. Это может включать в себя использование логина и пароля, двухфакторную аутентификацию или другие методы безопасности. После аутентификации пользователь получает доступ к интерфейсу управления, который может включать в себя графические панели управления, консоли командной строки или веб-интерфейсы. Пользователь может мониторить состояние оборудования, выполнять задачи управления, такие как перезагрузка,

обновление программного обеспечения или изменение конфигурации. Система может поддерживать автоматизацию задач и создание скриптов для выполнения повторяющихся задач или реагирования на определенные события.

Такие системы отлично работают на производстве, в отличие от решений, ориентированных на умные дома, но не имеют гибкости и интерактивного подхода в своем основании. Необходимо рассмотреть недостатки различных систем, чтобы синтезировать проблему, решение которой нужно предоставить.

1.1.5 Недостатки существующих систем

В рамках текущей работы предлагаю рассмотреть похожие существующие решения с точки зрения их недостатков:

- схожие решения не всегда являются проприетарными, а принадлежат конкретному вендеру;
- как правило такие системы разработаны под конкретное оборудование и требуют его наличие, что делает их абсолютно негибкими;
- в рамках политики импортозамещения определенных, к примеру, государственных, компаний некоторые инструменты могут не подойти, так как не являются отечественными;
- инструменты, ориентированные на интернет вещей, используют такие протоколы, как MQTT, что может не подойти для менее «умных» устройств, которые используются в промышленности;
- не все системы предоставляют гибкую настройку связи оборудования и интерфейса, а предлагают ограниченные способы это сделать, что может повлиять на конечный пользовательский опыт.

Некоторые из этих недостатков могут быть критическими в рамках тех условий, в которых проектируется и разрабатывается наша система, потому необходимо учитывать различные детали, создавая гибкий интерфейс для реализации программного комплекса взаимодействия со стендовым оборудованием.

1.1.6 Технологии, реализующие поставленные задачи

Для создания абстрактной реализации программного комплекса, способного работать с различными стендами и предоставлять удаленный доступ к техническим устройствам, необходимо рассмотреть несколько ключевых аспектов.

Во-первых, важно определить, какие физические объекты и пространство будут использоваться пользователями для взаимодействия с продуктом. Это может включать в себя использование ноутбука или смартфона.

При разработке программного комплекса важно также учитывать вопросы взаимодействия, такие как размеры и форма элементов интерфейса, которые должны быть достаточно разумными для взаимодействия. Это помогает стратегически думать о каждом элементе, используемом в продукте. Кроме того, важно использовать знакомые или стандартные форматы, чтобы упростить и улучшить обучаемость продукта

За основу взаимодействия с пользователем в системе выбрана интеграция с мессенджером Telegram через специализированного бота, что обусловлено популярностью этой платформы и возможностью реализации удобного и эффективного интерфейса и голосового помощника. Сложная логика взаимодействия системы с пользователем и стендом требует проработки алгоритмов обработки команд и сценариев работы, а также создания административной панели для настройки параметров работы с различными моделями оборудования и их конкретными характеристиками.

Также стоит отметить, что в рамках выбранной системы невозможно обойтись без базы данных. В дальнейшем в систему будет добавлен механизм сбора данных и их аналитики, что делает использование баз данных еще более необходимым. Таким образом, построенный дашборд будет удобным и понятным дополнением к общей среде клиента.

1.2 Описание методов обмена данными с промышленными устройствами

В основе анализа предметной области лежит обзор существующих программно-технических решений для интерактивного управления техническими устройствами и методов обмена данными с промышленными устройствами, в том числе с использованием сетевого протокола TCP/IP и стандарта OPC. Такой подход позволяет точно определить основные требования к разрабатываемой системе и принципы ее построения, обеспечивая надежное и безопасное управление стендовым оборудованием на расстоянии.

1.2.1 TCP/IP модель в производственных стендах

TCP (Transmission Control Protocol) играет ключевую роль в обеспечении надежной передачи данных между процессами в сети. Этот протокол используется в различных областях, включая производственные устройства, благодаря его способности обеспечивать надежность, управление потоком и последовательность пакетов.

TCP обеспечивает надежную передачу данных, используя механизмы обнаружения ошибок, повторную передачу потерянных пакетов и упорядочивание пакетов для гарантии доставки данных без ошибок и в правильном порядке. Чтобы предотвратить перегрузку получателя данными, TCP использует методы управления потоком для регулирования скорости передачи данных. TCP используется для организации данных таким образом, чтобы обеспечить безопасную передачу между сервером и клиентом, гарантируя целостность данных, передаваемых по сети.

Так как протокол поддерживается большим количеством устройств, использующихся на производстве, то выбор, сделанный в пользу его использования будет очевидным.

В рамках текущей задачи будет широкое использование протокола TCP.

1.2.2 Семейство HTTP протоколов

HTTP (Hypertext Transfer Protocol) и HTTPS (Hypertext Transfer Protocol Secure) являются протоколами, используемыми для передачи данных в интернете. HTTP позволяет обмениваться информацией между различными системами, передавая данные по сети. В отличие от HTTP, HTTPS обеспечивает защищенное соединение между веб-серверами и браузерами при передаче данных, используя цифровые протоколы безопасности, такие как SSL (Secure Sockets Layer) или TLS (Transport Layer Security), для шифрования и проверки данных.

В контексте разработки бэкенд приложений, которые система будет содержать, использование HTTP протокола является стандартом, обеспечивающим удобное взаимодействие между клиентом и сервером.

1.3 Описание используемого в ВКР стендового оборудования

Стендовое оборудование, используемое для реализации системы, в целом, не является конкретным, а лишь обладает некоторыми требованиями, необходимыми для создания интерфейса.

В рамках выпускной квалификационной работы будет рассматриваться оборудование, содержащее в своем составе ПЛК Omron nj301-1100, имеющий некоторый набор характеристик, соответствующих, тем, что представлены в требованиях. Внешний вид ПЛК представлен на Рисунке 1.4

Рисунок 1.4 — ПЛК Omron NJ301-1100

Подробные требования к системе представлены в Подразделе 1.4.

1.4 Определение требований к разрабатываемой ИУС

Требования к информационной системе (ИС) можно разделить на функциональные, нефункциональные, информационные и аппаратные. Для формирования функциональных требований к ИС, мы можем опираться на анализ уже существующих решений в данной области. Ниже в Таблице 1.1 представлены функциональные требования к системе.

Таблица 1.1 — Функциональные требования к ИУС

Требование	Механизм исполнения требований
Управлять конкретным стендом посредством нажатия команд	Набор команд, каждая из которых запускает определенный сценарий на стороне сервера
Сбор данных для аналитики	Отправление запросов о действиях пользователей в базу данных со стороны сервера
Наличие голосового помощника	Реализация озвучивания сообщений при помощи инструмента text-to-speech на стороне сервера
Создание сценариев	Создание сценариев из имеющихся конфигураций на стороне бота
Возможность настраивать функционал бота через админ-панель	Создать админ-панель так, чтобы она гибко настраивала каждый конкретный инстанс бота
Возможность изменять функционал бота через админ панель	Создать админ-панель так, чтобы она могла изменять наличие голосового помощника в боте.
Конфигурация переменных, которые встречаются в LD при помощи админ-панели	Добавить базовый функционал настройки конфигурации связи бота и стенда в админ-панель

Окончание Таблицы 1.1

Требование	Механизм исполнения требований
Возможность привязаться к любому боту, созданному в Telegram	Добавить в админ-панель возможность привязки к боту через установку токена
Отображать текущую информацию о стенде на дашборде	Добавить на дашборд слоты с отображением текущей информации по стенду
Отображать временной ряд активности стенда на дашборде	Добавить на дашборд слоты с отображением временного ряда активности стенда

В Таблице 1.2 представлены нефункциональные требования к ИУС

Таблица 1.2 — Нефункциональные требования к ИУС

Требование	Механизм исполнения требований
Производительность	Быстрая обработка и анализ данных в режиме мягкого реального времени
Надежность	Точное собирание и обработка данных, снижение вероятности потери информации
Доступность	Гарантия непрерывной работы системы, уменьшение времени простоя
Удобство использования	Интерфейс, легкий для понимания и использования

Так как система предполагает работу с различными данным, следует составить информационные требования. Они представлены в Таблице 1.3.

Таблица 1.3 — Информационные требования к ИУС

Требование	Механизм исполнения требований
Сбор данных	Сбор данных должен происходить в условиях мягкого реального времени в небольших объемах
Обработка данных	Данные должны обрабатываться на сервере, выводиться в удобном формате после преобразования
Хранение данных	Данные хранятся на едином сервере реляционной БД, разделенном по логике на таблицы

Аппаратные требования к системе предоставляются отдельно к стендовому и к серверному оборудованию (представлены в Таблицах 1.4-1.5).

Таблица 1.4 — Аппаратные требования к серверному оборудованию

Требование	Описание
Процессор	64 bit процессор с поддержкой Second Level Address Translation (SLAT), поддержка минимум 4 потоков
Оперативная память	Минимум 8GB памяти
Жесткий диск	Минимум 500 МБ SSD диск
Виртуализация	Поддержка аппаратной виртуализации на уровне BIOS
Доступ в Интернет	Ethernet порт и/или интерфейс Wi-Fi с пропускной способностью от 1000 Мб/с
Бесперебойное питание	Источник бесперебойного питания (ИБП)

Таблица 1.5 — Аппаратные требования к ПЛК

Требование	Описание
Поддержка протоколов	Поддержка протокола TCP
Среда исполнения	Многозадачная
Языки программирования	IEC 61131-3 (LD, IL, ST, FBD, SFC)

1.5 Постановка цели и задач ВКР

Целью ВКР является создание универсальной и гибкой информационно-управляющей системы, способной эффективно интегрироваться с различными типами стендового оборудования и обеспечивать качественное взаимодействие между пользователем и техническими устройствами. Важность системы усиливается в контексте современных требований к инженерным и производственным процессам, где оперативность, точность и адаптивность играют решающую роль.

Задачи ВКР:

- спроектировать архитектуру программного комплекса интерактивного взаимодействия со стендовым оборудованием;
- спроектировать и реализовать базу данных для дальнейшей работы в программной среде;
- разработать программного комплекс интерактивного взаимодействия со стендовым оборудованием.

2 ПРОЕКТНЫЙ РАЗДЕЛ

2.1 Моделирование элементов ИУС

2.1.1 Концептуальная модель системы

Для создания концептуальной модели системы, была выбрана нотация графического моделирования IDEF0. По описанному технологическому процессу была составления модель в нотации IDEF0 (Рисунок 2.1-2.2).

На контекстной диаграмме (Рисунок 2.1) показаны входные и выходные потоки данных и материалов, нормативные документы и используемое оборудование. В качестве материальных входов в системе используется стендовое оборудование. Информационными входами являются шаблон интерфейса, данные о конфигурациях, пользовательский сигнал. В качестве управления используются ГОСТы, предъявляемые к ИУС в целом и к созданию конкретных ее частей, требования к конфигурации системы, которые накладывают ограничения на вводимые значения. Механизмами служат персонал и различное оборудование: датчики, приводы, серверное оборудование, устройства ввода-вывода и т.д. Выходом системы является настроенное и готовое к эксплуатации стендовое оборудование, а также настроенный интерфейс и обработанные данные, полученные по результатам выполнения сценария.

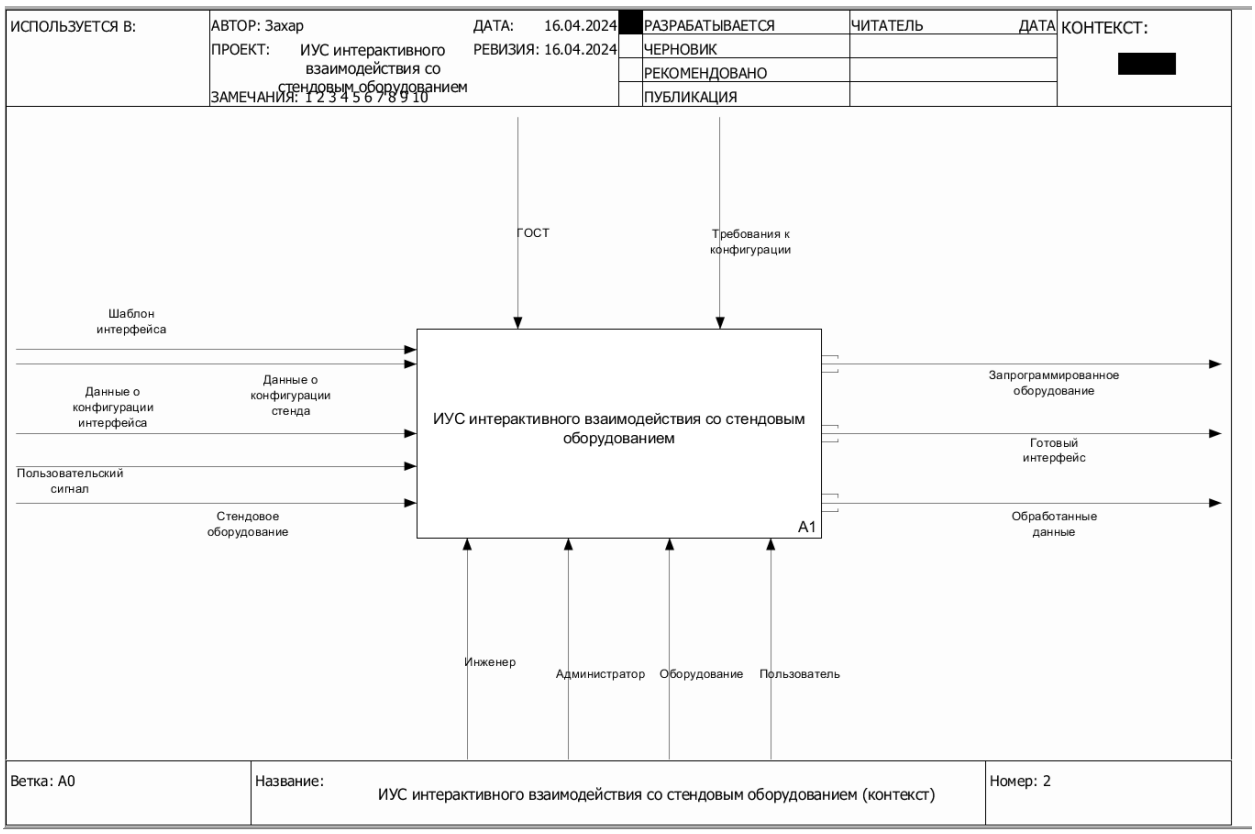

Рисунок 2.1 — Контекстная диаграмма процесса в нотации IDEF0

Сотрудники разделены по следующему принципу:

- инженер. В его компетенции входит конфигурация конкретного стенда, поддержка его работы;
- администратор. Создает, запускает и настраивает бота, поддерживает его работу;
- конечный пользователь. Посылает команды стенду через бота, тем самым выполняя сценарии.

На первом уровне декомпозиции контекстной диаграммы (Рисунок 2.2) присутствуют блоки: «Конфигурация стенда», «Конфигурация бота», «Выполнение сценариев».

В процессе конфигурации стенда на вход подается конфигурация стенда (как информационный поток) и само стендовое оборудование (как физический поток). Выходом являются настроенный, работающий стенд, применяемая текущая конфигурация проекта. Инженер подключается к ПЛК и на языке стандарта LD настраивает значения внутренних переменных и их названия для дальнейшей интеграции с ботом при помощи гибкого сервиса.

В процессе конфигурация бота участвует администратор. Он создает бота через Telegram, запускает, его и через заранее им админ панель настраивает его конфигурацию: привязывая команды бота к конкретным сценариям и переменным. На входе в блок подаются исходные данные о конфигурации и шаблон интерфейса админ панели и бота, на выходе появляются данные о применяемой, выбранной конфигурации бота и готовый настроенный интерфейс бота.

В процессе выполнения сценария пользователь выбирает, какой сценарий ему запустить, после чего его сигнал обрабатывается на сервере, передается на стенд, и сценарий выполняется. После чего данные о действии заносятся в базу данных для логирования и последующей аналитики.

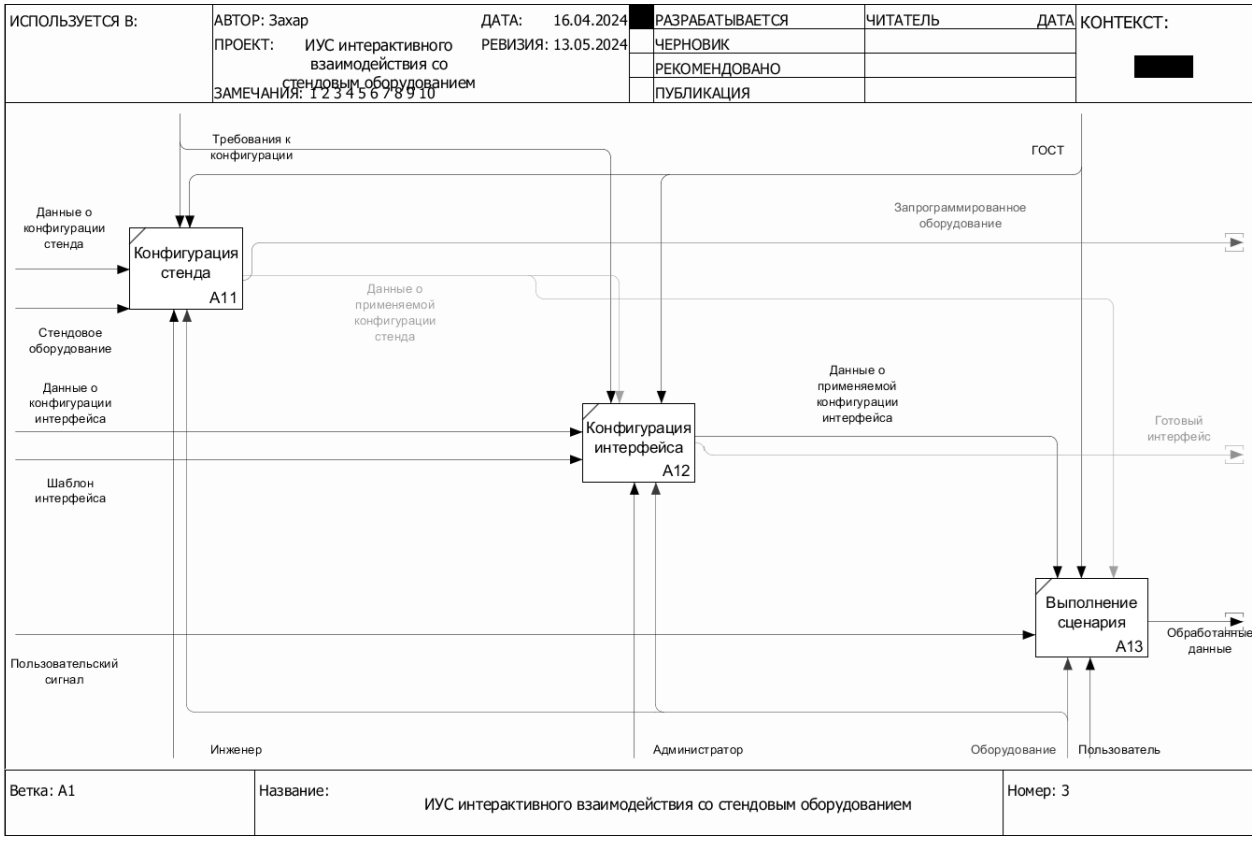

Рисунок 2.2 — Декомпозиция контекстной диаграммы процесса в нотации IDEF0

2.1.2 Динамическая модель в нотации BPMN

Для более точного проектирования системы, необходимо рассмотреть последовательность взаимодействия пользователей с ней. Составим диаграмму в нотации BPMN (Рисунок 2.3).

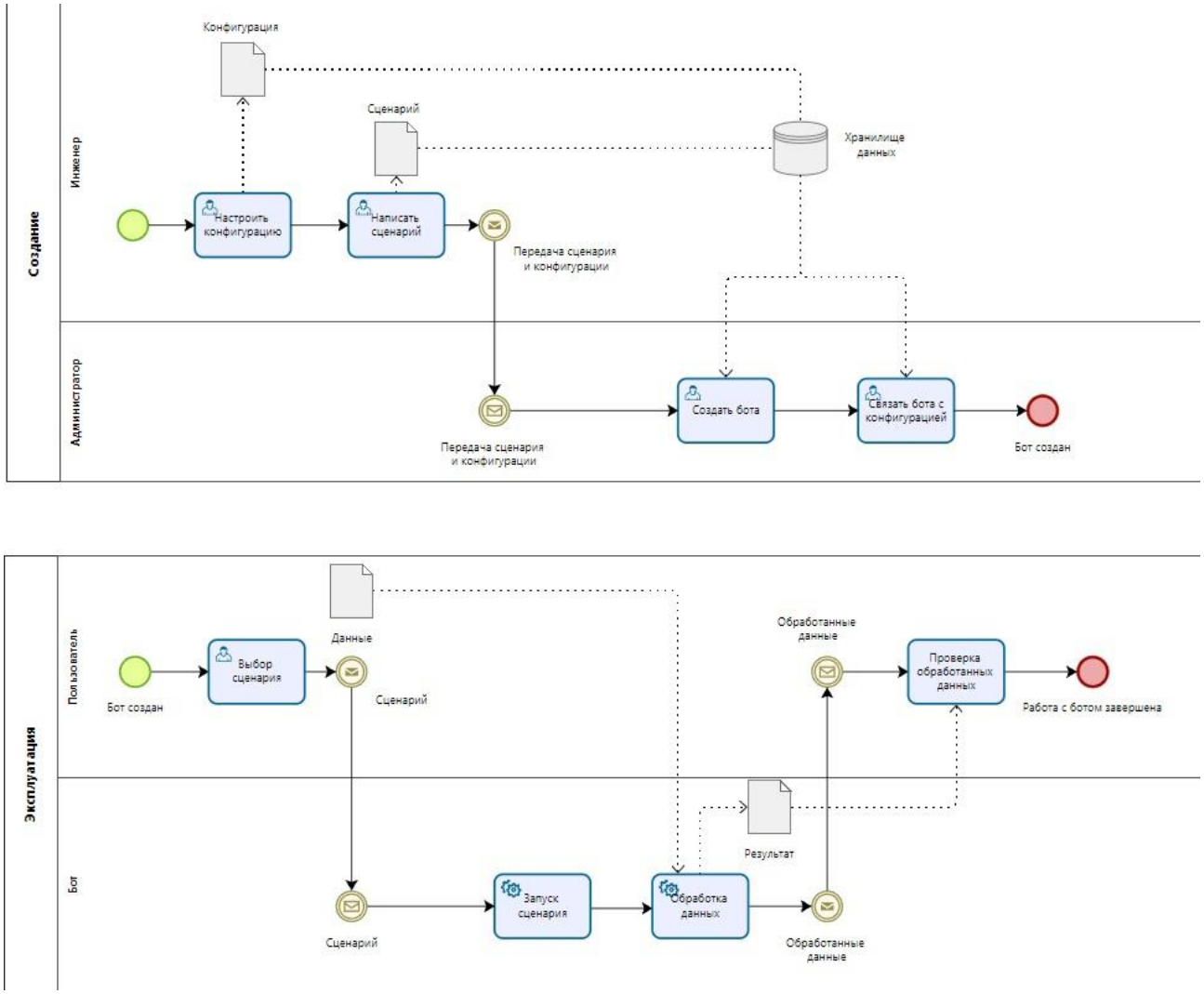

Рисунок 2.3 — Диаграмма в нотации BPMN

На изображении представлена диаграмма BPMN, которая описывает бизнес-процесс, связанный с созданием и использованием ботов. Диаграмма разделена на четыре основных линии (пула), которые представляют различные роли или акторы, участвующие в процессе: «Администратор», «Бот» и два пула без явных названий, относящихся к действиям пользователя и обработке данных соответственно.

Верхняя линия (пул «Администратор») показывает, что администратор начинает с настройки конфигурации, затем переходит к анализу сценариев и

пересылке сценариев и конфигурации. После этого в процесс вступает хранилище данных. Завершается процесс созданием бота с конфигурацией.

Вторая линия (пул «Бот») показывает, что бот начинает с выбора сценария после его создания, затем сценарий используется, и завершается запуском сценария.

Третья линия (без названия) иллюстрирует взаимодействие пользователя с ботом и процесс обработки данных: пользователь получает результат, после чего данные обрабатываются.

Нижняя линия (также без названия) относится к обработке данных и включает проверку обработанных данных, после чего работа с ботом завершается.

На диаграмме присутствуют стандартные элементы BPMN, такие как события (зеленые круги), задачи (прямоугольники с закругленными углами), решения (ромбы), а также соединения между элементами, включая последовательные потоки (сплошные стрелки) и сообщения (пунктирные стрелки). Также присутствует база данных (символ цилиндра), которая используется для хранения данных в процессе.

2.2 Разработка архитектуры ИУС

2.2.1 Диаграмма компонентов

Для представления основных модулей системы, взаимодействия между ними, передаваемыми данными, необходимо разработать диаграмму компонентов (Рисунок 2.4).

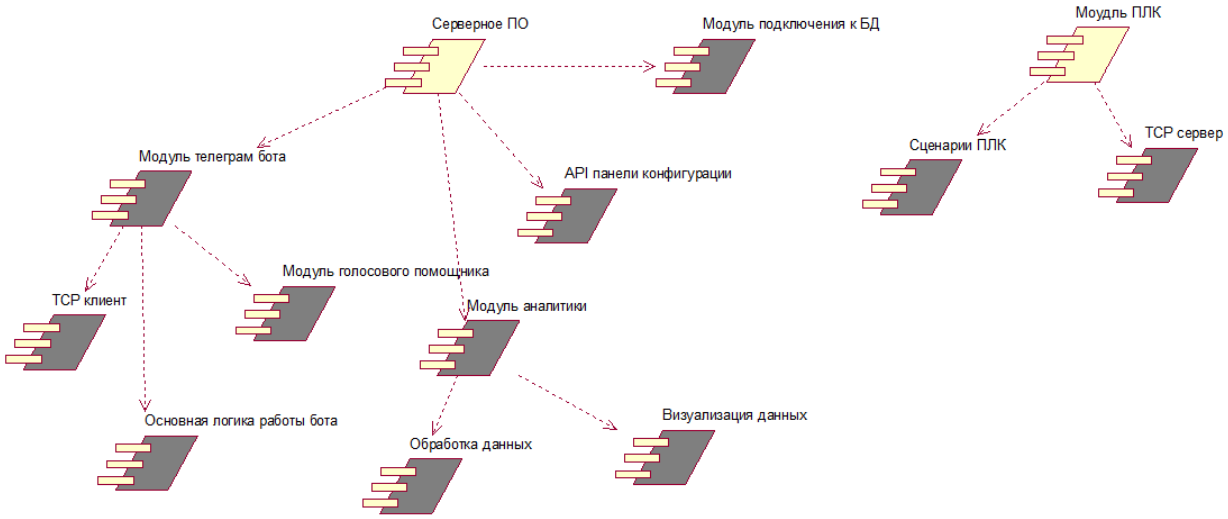

Рисунок 2.4 — Диаграмма компонентов ИУС

Диаграмма содержит следующие модули:

- модуль бекенд части Telegram. Модуль отвечает за работу с API Telegram бота;
- модуль админ-панели — бекенд и фронтенд части админ-панели, необходимые для ее работы;
- модуль анализа. Модуль производит анализ, выполняет обработку и визуализацию данных;
- модуль работы с ПЛК включает в себя различные низкоуровневые сценарии.

2.2.2 Общая архитектурная схема

На Рисунке 2.5 изображена схема интерактивного взаимодействия пользователя и стендового оборудования, структурированная на несколько основных компонентов и взаимодействующих между собой элементов.

Пользовательская сторона это Telegram клиент: представляет собой приложение мессенджера Telegram, через которое пользователь может взаимодействовать с системой.

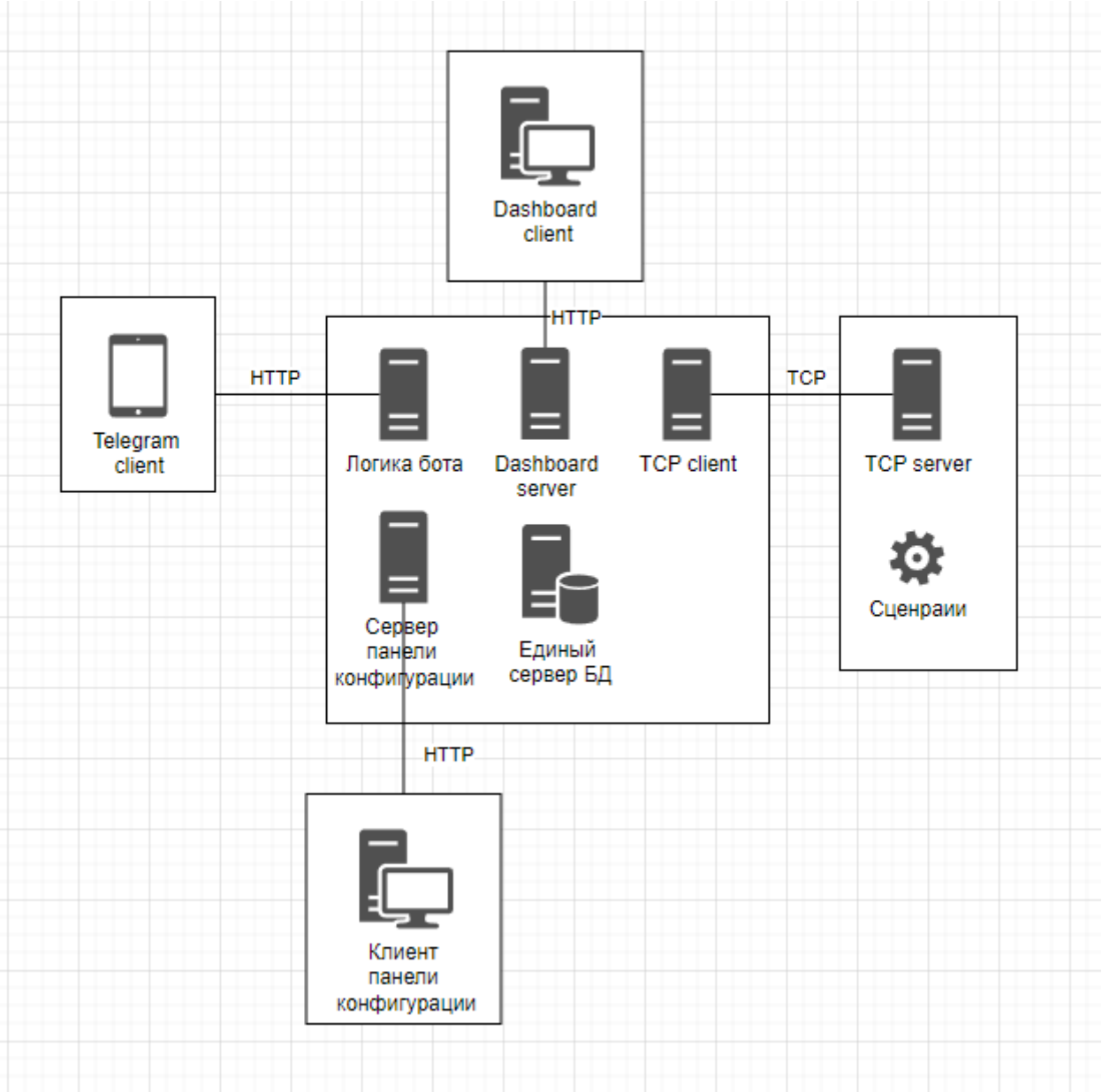

Рисунок 2.5 — Общая архитектура ИУС

Серверная сторона включает в себя:

- БД (база данных). БД хранит данные, необходимые для работы всей системы;
- бот. Бот отвечает за обработку команд и запросов, отправленных пользователем через Telegram клиент;
- admin панель. Серверная часть административной панели обрабатывает запросы от компьютера инженера и управляет стендовым оборудованием;
- сервис аналитики – компонент, предназначенный для сбора, обработки и анализа данных.

Стенд состоит из:

- ПЛК (программируемый логический контроллер). ПЛК контролирует и управляет техническими процессами на стенде;
- оборудование стенда общается с ПЛК по протоколу TCP/IP, который является стандартом для передачи данных между устройствами в сетях.

Также в схеме указаны:

- ПК инженера. Содержит фронтенд админ панели: графический интерфейс, через который инженер может мониторить и управлять процессами на стенде;
- ПК аналитика. Отображает информационную панель, показывающую ключевые показатели и метрики, полученные из сервиса аналитики.

Взаимосвязи между компонентами:

- HTTP (протокол передачи гипертекста) используется для взаимодействия между Telegram клиентом, сервисом аналитики, фронтендом и бэкендом админ панели;
- TCP/IP служит для связи между ПЛК и бэкендом бота.

В целом, данная схема позволяет пользователям использовать Telegram для отправки команд, контролировать аппаратуру на стенде через админ панель и анализировать данные через дашборд аналитики.

2.3 Разработка логической модели данных ИУС

Информация является ключевым аспектом в работе любой системы, создание хранилища данных необходимо для корректного функционирования ИУС. Перед реализацией БД важно спроектировать её, для этого создадим логическую модель БД (Рисунок 2.6).

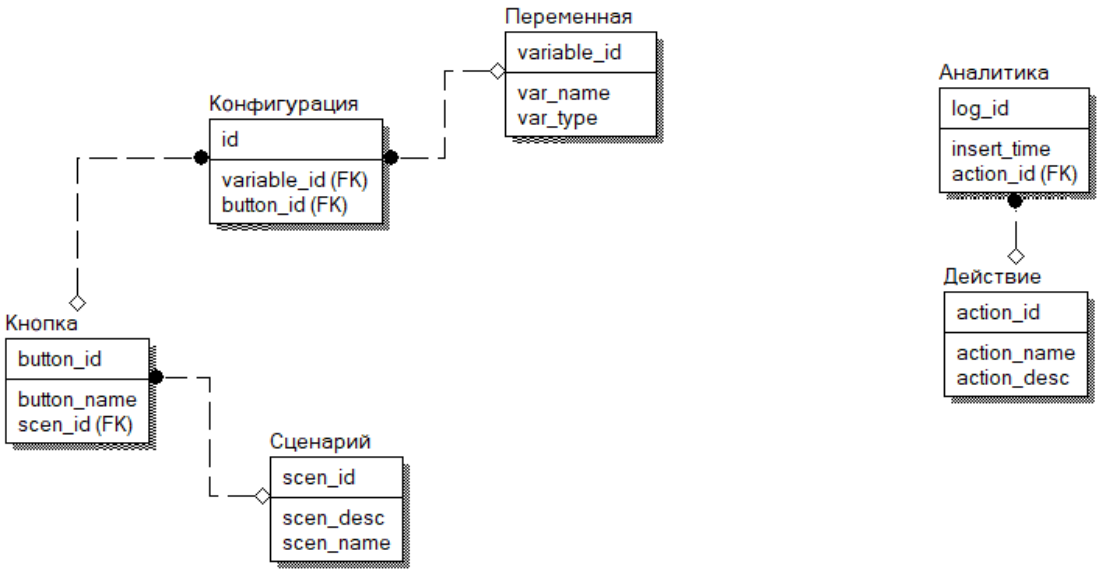

Рисунок 2.6 — Логическая модель базы данных ИУС

На данной ER-диаграмме (Entity-Relationship diagram, диаграмма сущность-связь) представлены пять сущностей, которые участвуют в процессе интерактивного взаимодействия пользователя со стендовым оборудованием:

- переменная. Содержит информацию о настроенной администратором переменной, ее тип и название;
- конфигурация. Является связующей таблицей между переменной и кнопкой (или командой) со связью многие-ко-многим;
- кнопка (или команда). Содержит id кнопки, привязанное к конкретному Telegram боту, необходимой для отправки сигнала;
- сценарий. Конкретный use-case, к которому привязана текущая команда;
- действие. Тип действия, вызванного пользователем. Таблица необходима для удобства расширяемости БД;
- аналитика. Содержит в себе данные о том, что пользователь выполнил во временной перспективе.

Связи между сущностями обозначены сплошными линиями с маркерами отношений (птички или линии), указывающими на тип связи (один-к-одному, один-ко-многим и т.д.). Например, Конфигурация связана с «Переменной» и

«Кнопкой», при этом предполагается, что конфигурация может включать в себя множество переменных и кнопок. «Кнопка» связана со «Сценарием», указывая на то, что один сценарий может быть связан с многими кнопками. «Действие» связано с «Аналитикой», что отражает, что одно действие может генерировать множественные аналитические записи.

Схема также показывает, что процесс включает в себя определение сценариев взаимодействия, настройку переменных и конфигурации кнопок для взаимодействия, а также фиксацию результатов действий в аналитической системе.

3 ПРОГРАММНЫЙ РАЗДЕЛ

3.1 Выбор платформ и средств разработки

Для реализации необходимого функционала нужно выбрать, какие инструменты будут использоваться при разработке ПО. Для этого необходимо учесть, какие задачи стоят на данный момент и выбрать те решения, которые наиболее хорошо подходят для текущей системы в соответствии с ее требованиями.

3.1.1 Выбор модели ПЛК для стендового оборудования

В рамках данной работы применяется уже настроенное демонстрационное оборудование, описание которого представлено в Подраздел 1.4.

ПЛК Omron NJ301-1100, используемый на стенде, полностью удовлетворяет всем поставленным требованиям, а именно:

- является программируемым;
- поддерживает стандарт языков IEC 61131-3;
- поддерживает сетевое взаимодействие по протоколу TCP.

3.1.2 Выбор средств разработки для стендового оборудования

Для ПЛК, поддерживающего стандарт IEC 61131-3, есть выбор языка программирования для разработки. В данном случае было решено остановиться на языке LD, так как он довольно прост в освоении и понятен для большинства инженеров.

3.1.3 Выбор ПК для реализации серверного компонента

Существует большое количество различных ПК с разными аппаратными характеристиками, стоит разобраться, какие именно лучше всего подходят для данной системы.

Аппаратные требования к серверному оборудованию описаны в Подразделе 1.4. Требования синтезированы в соответствии с тем, что на ПК, который будет служить сервером для веб части системы и реализовывать всю ее логику, необходимо установить Docker. В следствии этого, необходимо ориентироваться на его минимальные требования.

В качестве ПК на этапе разработки и отладки будет использован стационарный компьютер, находящийся в одном помещении с ПЛК.

В качестве финального варианта будет использован портативный компьютер на базе процессора Raspberry Pi именно на нем и будет работать логика системы, не привязанная к стендовому оборудованию.

3.1.4 Выбор операционной системы для реализации серверного компонента

В случае с ПК разработки и отладки, была выбрана ОС Windows, благодаря ее удобству использования в качестве среды разработки. Система не лишена каких-либо проблем, но на данном этапе она подходит под конкретные задачи.

Для основы системы, на которой будут работать компоненты была выбрана система Linux. Это обусловлено, в первую очередь, тем, что ее удобно администрировать, она потребляет меньшее количество ресурсов в сравнении с системами аналогами, что может быть очень важно, учитывая выбранное оборудование для нашей системы. Также это систему несложно поставить на компьютер, и некоторые дистрибутивы имеют отличную оптимизацию под выбранные далее программные решения.

В качестве дистрибутива линукс для финального варианта была выбрана Ubuntu. Этот дистрибутив основан на Debian, имеет удобный пакетный менеджер, со стабильными версиями пакетов и является довольно удобным для администрирования, также работает «из коробки».

3.1.5 Выбор средств разработки отдельных компонентов серверной части

Для разработки Telegram бота лучше всего воспользоваться языком программирования Python и стабильной версией его библиотеки telebot. Существует также аналог библиотеки для написания Telegram ботов, такой, как aiogram, но в данный случае его использование не обосновано в следствии отсутствия большой нагрузки на сервер. Он поддерживает асинхронный подход, который усложняет процесс написания и чтения кода и нужен только в тех случаях, когда не хватает производительности сервера.

Также выбор языка Python для данной задачи вызван удобным вариантом TCP подключения, которое можно интегрировать внутрь сервисов.

Для реализации панели конфигурации, при выборе между десктоп приложением и веб-интерфейсом, выбор был сделан в пользу последнего. Это обеспечит гибкость системы, удобство пользования, в следствии возможности получить доступ к панели не с конкретного ПК, а с любого устройства, наделенного браузером, находящегося в локальной сети.

Язык программирования, отлично подходящий для быстрого написания бекенда небольших веб-приложений – Python. В сравнении с другими языками, он дает наименьший порог входа для разработчика, является удобным с точки зрения написания кода, но довольно медленным в следствии его интерпретируемости, что в данном случае не так важно.

Из популярных фреймворков для написания бекенд части приложения наиболее удобным и гибким является фреймворк FastAPI. По сравнению с аналогами на Python, он дает наибольшие возможности и является самым современным из них, он также дает возможность простой интеграции с другими сервисами, что будет необходимо в данной системе.

Для реализации голосового помощника, повышающего интерактивность системы, был выбран язык Python, он содержит в себе большое количество стабильно работающих, оптимизированных на низком уровне и современных библиотек для обработки данных и созданию моделей машинного обучения.

Для сервиса аналитики работы системы был выбран Python по тем же причинам.

3.1.6 Выбор СУБД для системы

Для любой задачи при проектировании системы хранения данных встает выбор между SQL и NoSQL подходами. SQL подход дает наибольшее удобство работы с данными, в тех случаях, когда необходимо использование OLTP решений. Также он позволяет писать запросы на языке SQL, что оптимизирует работу системы. Именно поэтому выбор был сделан в пользу SQL подхода.

Среди большого количества различных SQL БД, было решено использовать PostgreSQL. Эта БД является гибкой с точки зрения хранения различных типов данных, является безопасной и имеет удобные и быстрые инструменты для администрирования и настройки.

3.2 Реализация базы данных

База данных для данной системы является необходимостью, так как конфигурации и различная служебная информация будут храниться на ее сервере.

Далее будет описан процесс создания контейнера с базой данных и создания самой базы данных.

3.2.1 Создание сервера базы данных

Популярным решением в наши дни является развертка базы данных внутри docker контейнера. Это дает возможность легко запускать сервер базы данных и разграничить его с другими сервисами.

Был написан Dockerfile, который используется при запуске контейнера, запуская готовый образ с определенными параметрами, такими как:

- порт;
- имя контейнера;
- имя пользователя БД;

- пароль для пользователя;
- название БД.

Код создания и поднятия базы данных представлен в Листингах 3.1-3.4.

Листинг 3.1 — Пример docker-compose файла для запуска БД

```
version: '3.9'

services:
  postgres:
    image: postgres:14.3-alpine
    ports:
      - "8765:5432"
    env_file:
      - .env
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

В данном случае используется уже готовый образ PostgreSQL, открывается порт 5432 для подключения к БД, создается volume для сохранения данных в случае перезапуска контейнера, переменные окружения, такие как пользователь, пароль и имя БД получаются из .env файла.

Листинг 3.2 — Пример .env файла для запуска БД

```
POSTGRES_USER=user
POSTGRES_PASSWORD=pass
POSTGRES_DB=postgres
```

По итогу создания базы данных видим готовый контейнер, к которому уже можно подключаться. Пример запуска контейнера представлен на Рисунках 3.1-3.2

```
(venv) PS E:\13axap\8 сем\Диплом\code\prototype\db> docker-compose up
[+] Running 1/0
 - Container db-postgres-1  Created
Attaching to db-postgres-1
db-postgres-1  |
db-postgres-1  | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-postgres-1  |
db-postgres-1  | 2024-05-09 15:55:21.417 UTC [1] LOG:  starting PostgreSQL 15.3 (Debian 15.3-1.pgdg120+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 12.2.0-14) 12.2.0, 64-bit
db-postgres-1  | 2024-05-09 15:55:21.417 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
db-postgres-1  | 2024-05-09 15:55:21.417 UTC [1] LOG:  listening on IPv6 address "::", port 5432
db-postgres-1  | 2024-05-09 15:55:21.434 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-postgres-1  | 2024-05-09 15:55:21.452 UTC [29] LOG:  database system was shut down at 2024-05-09 15:38:41 UTC
db-postgres-1  | 2024-05-09 15:55:21.466 UTC [1] LOG:  database system is ready to accept connections
```

Рисунок 3.1 — Запуск докер контейнера с базой данных

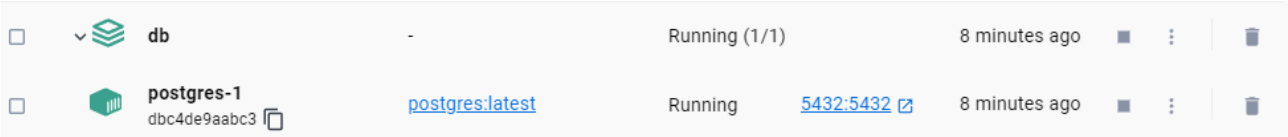

Рисунок 3.2 — Работа докер контейнера с базой данных

3.2.2 Создание базы данных на сервере

Для создания базы данных был написан SQL скрипт, который запускается при создании базы данных, он был добавлен в Dockerfile, что позволяет не создавать ее вручную каждый раз

Данные, которые будут в БД содержат таблицы с конфигурациями бота и аналитику.

Листинг 3.3 — Часть SQL скрипта создания БД

```
DROP TABLE IF EXISTS Scenario;
CREATE TABLE Scenario (
    scen_id SERIAL,
    scen_desc TEXT,
    scen_name TEXT,
    PRIMARY KEY (scen_id)
);

INSERT INTO Action VALUES
(1, 'floor move', 'Move up to a specific floor'),
(2, 'light switch', 'Switch the light on a button');

DROP VIEW IF EXISTS analytics_log;
CREATE VIEW analytics_log AS
SELECT b.action_name, DATE_TRUNC('hour', insert_time) AS hour, COUNT(*) AS cnt
FROM Analytics AS a
INNER JOIN Action AS b
ON a.action_id = b.action_id
GROUP BY b.action_name, DATE_TRUNC('hour', insert_time)
ORDER BY hour;

DROP VIEW IF EXISTS analytics_log_distr;
CREATE VIEW analytics_log_distr AS
SELECT b.action_name, COUNT(*) AS cnt
FROM Analytics AS a
INNER JOIN Action AS b
ON a.action_id = b.action_id
GROUP BY b.action_name;
```

В данном скрипте помимо создания таблиц, происходит заполнение этих таблиц значениями, и создаются view (представления) для удобного анализа данных. Наличие view позволяет агрегировать данные на уровне БД, что позволяет упростить логику дашборда.

Полный скрипт представлен в Приложении А.

Далее необходимо добавить скрипт init.sql в docker контейнер.

Листинг 3.4 — Пример docker-compose файла для запуска и инициализации БД

```
version: '3.9'

services:
  postgres:
    image: postgres:14.3-alpine
    ports:
      - "8765:5432"
    env_file:
      - .env
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  postgres_data:
```

После чего этот скрипт можно запустить из базы данных, что, на самом деле, необязательно, так как он запускается при каждом старте docker контейнера. Результат запуска таблицы представлен на Рисунке 3.3-3.4.

```
# ls
bin  boot  dev  docker-entrypoint-initdb.d  etc  home  lib  lib32  lib64  libx32  media  mnt  opt  proc  root  run  sbin  sr
# cd docker-entrypoint-initdb.d
# ls
init.sql
# psql -f init.sql
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  role "root" does not exist
# psql -U postgres -d postgres -f init.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
#
```

Рисунок 3.3 — Инициализация БД

```
# psql -U postgres -d postgres
psql (15.3 (Debian 15.3-1.pgdg120+1))
Type "help" for help.

postgres=# \d
              List of relations
 Schema |     Name     |   Type   |  Owner
--------+--------------+----------+----------
 public | action       | table    | postgres
 public | analytics    | table    | postgres
 public | button       | table    | postgres
 public | config       | table    | postgres
 public | scenario     | table    | postgres
 public | users        | table    | postgres
 public | users_id_seq | sequence | postgres
 public | var          | table    | postgres
(8 rows)
```

Рисунок 3.4 — Проверка наличия таблиц в БД

Таким образом, таблицы были успешно созданы.

3.3 Разработка стендового компонента системы

3.3.1 Создание TCP сервера для сетевого взаимодействия с системой

Работа с TCP в Sysmac Studio на языке LD (Ladder Diagram) включает в себя несколько ключевых шагов и понятий. В Sysmac Studio, это можно реализовать через специальные функции и блоки, предназначенные для работы с TCP.

1. Настройка TCP. В Sysmac Studio для работы с TCP используются специальные блоки для отправки и получения данных. Эти блоки позволяют легко интегрировать систему с другими устройствами;
2. Использование Блоков. Перед началом работы с Modbus TCP в Sysmac Studio, необходимо настроить TCP сервер или клиент в системе. Это может быть выполнено через встроенные средства Sysmac Studio или через сторонние программы, которые поддерживают TCP. В данном случае это будет выполнено на стороне сервере при помощи библиотеки языка программирования Python;
3. Тестирование и Деплой. После написания кода на языке LD, его необходимо тестировать, чтобы убедиться в корректности работы. Это может включать в себя проверку связи с устройством, чтение и запись данных, а также обработку ошибок.

Часть кода на LD, отвечающий за указанные действия представлена на Рисунке 3.5.

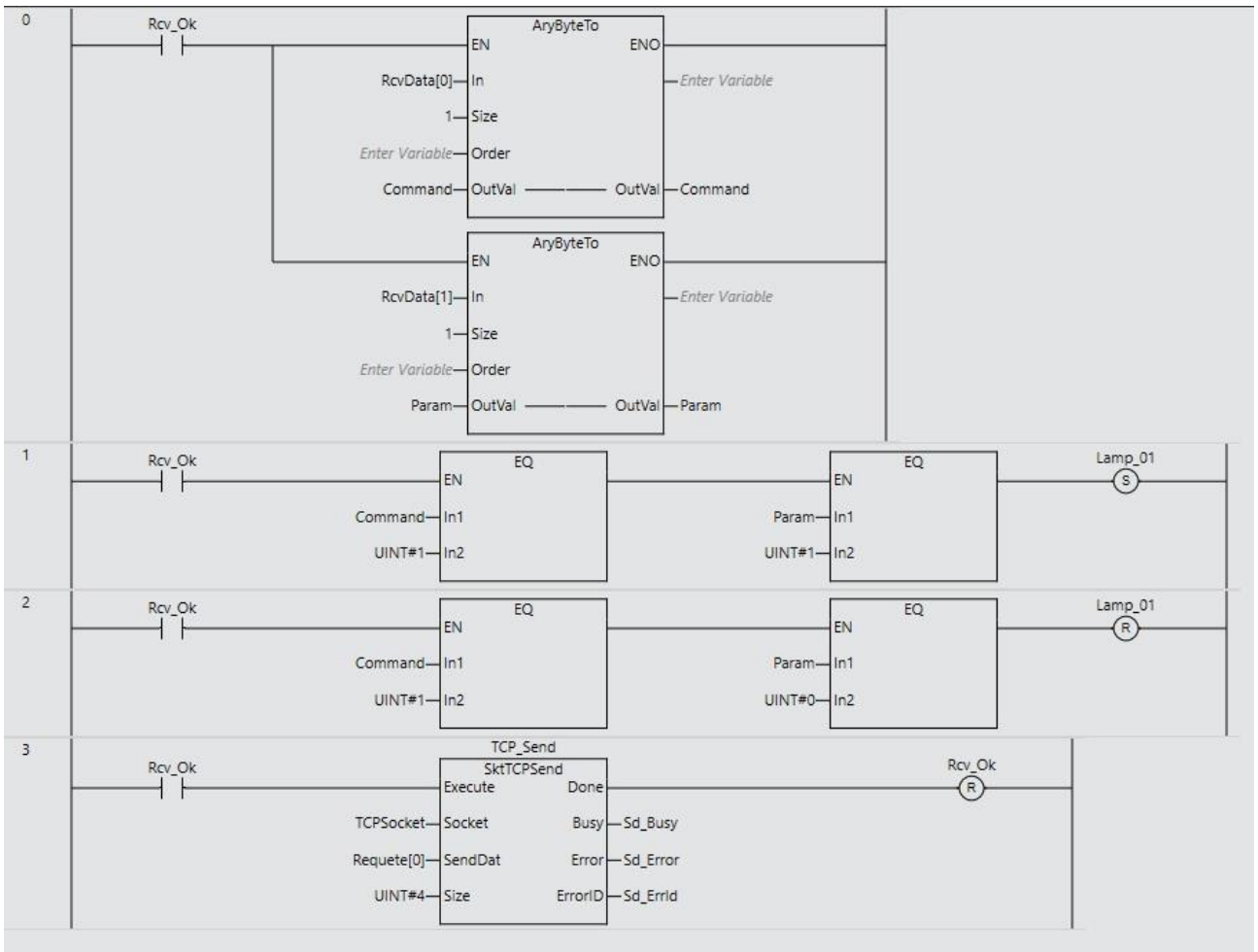

Рисунок 3.5 — Блоки TCP соединения в Sysmac Studio

Полный код создания TCP сервера на ПЛК представлен в Приложении Б.

3.3.2 Написание простого сценария на стендовом оборудовании

Сценарий, написанный на языке LD, будет состоять из простых действий, которые будут выполняться на стенде. Стенд будет обладать собственной логикой и обрабатывать поступающие на TCP сервер запросы, доставать из TCP пакета команду и выполнять простые действия при условии ее наличия.

Например, стенд получил команду на загорание лампочки 1, переменная, отвечающая за эту команду, выступает в роли обмотки, и, если ее значение истинно, то выполняется блок кода, написанного на языке LD. На Рисунке 3.6 представлен участок кода

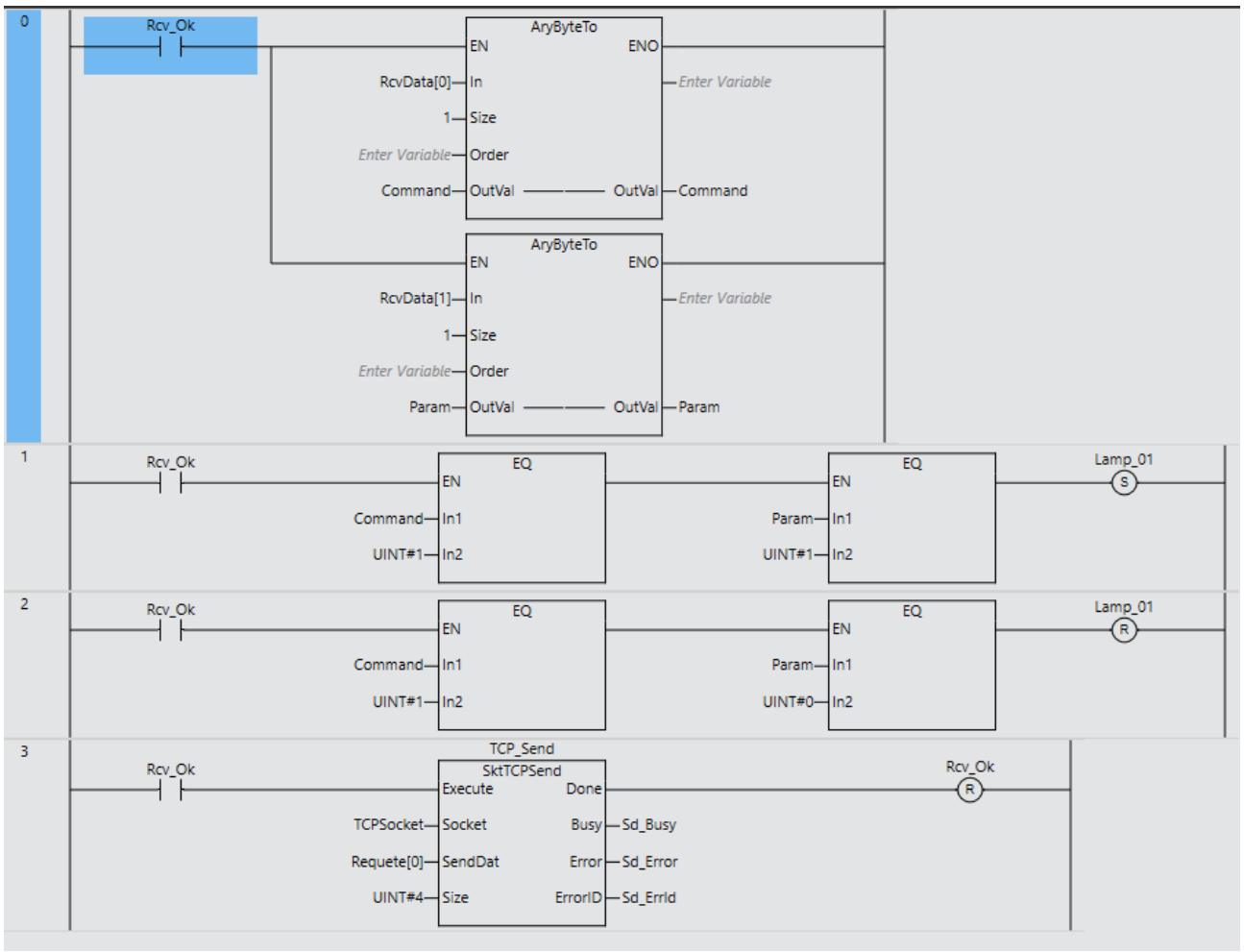

Рисунок 3.6 — Сценарий в Sysmac Studio

Данные, получаемые из БД и отправляемые на TCP сервер состоят из 2 байтов: первый отвечает за адрес команды, второй отвечает за ее параметр. Данный участок получает информацию о команде и параметрах при помощи преобразования в целые числа, после чего выполняет необходимые действия.

3.4 Реализация серверного компонента системы

3.4.1 Создание Telegram бота

Для начала необходимо создать Telegram бота через BotFather внутри Telegram, после чего написать основной функционал работы бота. На Рисунке 3.7 представлен пример создания бота.

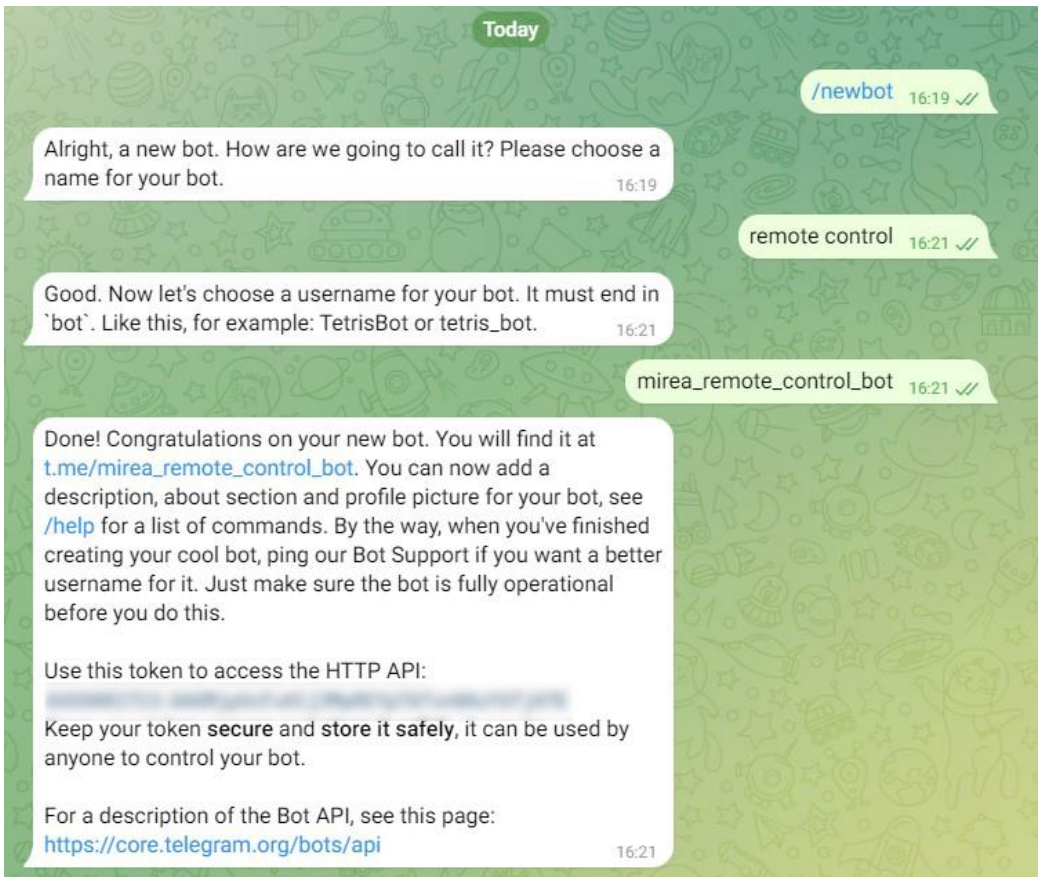

Рисунок 3.7 — Создание Telegram бота

Теперь можно подключаться к боту, используя токен, который предоставляет BotFather ля подключения к API Telegram. В данном случае приложение выступает как сервер для бота и как клиент для API Telegram. Внутри библиотеки находятся простые и интуитивно понятные методы, содержащие в себе функционал для отправки HTTP запроса на сервер Telegram.

При написании бота для начала необходимо создать виртуальное окружение. Создание и запуск бота представлена на Рисунке 3.8-3.10.

```
PS E:\13axap\8 сем\Диплом\code\prototype\tg> python -m venv venv
PS E:\13axap\8 сем\Диплом\code\prototype\tg> .\venv\Scripts\Activate.ps1
(venv) PS E:\13axap\8 сем\Диплом\code\prototype\tg>
```

Рисунок 3.8 — Создание виртуального окружения

Теперь нужно установить необходимые пакеты.

```
(venv) PS E:\13axap\8 сем\Диплом\code\prototype\tg> pip install pyTelegramBotAPI
Collecting pyTelegramBotAPI
  Downloading pytelegrambotapi-4.17.0-py3-none-any.whl (240 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 240.6/240.6 kB 2.5 MB/s eta 0:00:00
Collecting requests
  Using cached requests-2.31.0-py3-none-any.whl (62 kB)
Collecting charset-normalizer<4,>=2
  Using cached charset_normalizer-3.3.2-cp311-cp311-win_amd64.whl (99 kB)
```

Рисунок 3.9 — Установка необходимых пакетов

Данные действия будут выполняться для всех проектов, в которых используется Python.

Далее, выделив переменные окружения, нужно написать код самого бота, добавить туда код подключения к БД, из которой будут получены данные о конфигурации бота.

Наиболее удобным вариантом взаимодействия пользователя с системой являются кнопки действий, которые отвечают за отправку команд на другие сервисы.

Рисунок 3.10 — Пример интерфейса Telegram бота

Также в дальнейшем будет добавлен инструмент speech-to-text, который алгоритмически определить, какая команда ожидается от пользователя. Часть кода создания бота представлена на Листинге 3.5.

Листинг 3.5 — Часть кода работы боты

```
import telebot
from telebot import types

from config import TOKEN
from db import insert_action_data, get_command
from TCP import send_TCP_request
from speech_encoder import recognize

bot = telebot.TeleBot(TOKEN, parse_mode=None)


def act(button_name, param, call, action_type_id):
    insert_action_data(action_type_id)
    command = get_command(button_name, param)
    res = send_TCP_request(f'{command[0]}{command[1]}')
    bot.send_message(call.message.chat.id, res)


@bot.message_handler(commands=['light'])
def light(message):
    keyboard = types.InlineKeyboardMarkup(row_width=2)  # Создаем клавиатуру с
двумя кнопками в строке
    btn_1 = types.InlineKeyboardButton("Лампочка 1 выкл",
callback_data='button_3')
    btn_2 = types.InlineKeyboardButton("Лампочка 1 вкл",
callback_data='button_4')
    btn_3 = types.InlineKeyboardButton("Лампочка 3", callback_data='test')
    keyboard.add(btn_1, btn_2, btn_3)  # Добавляем кнопки на клавиатуру
    bot.send_message(message.chat.id, 'Выберите кнопку:', reply_markup=keyboard)
# Отправляем сообщение с клавиатурой


@bot.callback_query_handler(func=lambda call: True)  # Обработчик для callback-
запросов
def callback_inline(call):
    insert_action_data(1)
    if call.data == 'button_1':
        act(call.data, '01', call, 1)
    elif call.data == 'button_2':
        act(call.data, '02', call, 1)
    elif call.data == 'button_3':
        act(call.data, '00', call, 2)
    elif call.data == 'button_4':
        act(call.data, '00', call, 2)
    elif call.data == 'test':
        act(call.data, '00', call, 1)


bot.infinity_polling()
```

Весь код бота представлен в Приложении В.

3.4.2 Создание TCP клиента для сетевого взаимодействия с системой

В рамках данной работы необходимо создать и протестировать TCP соединение на одном хосте, создав отдельно клиентскую и серверную часть приложения и соединив их вместе. Для этого отлично подойдет встроенная в

Python библиотека socket. Пример создания TCP соединения представлен на Листингах 3.6-3.7.

Создаем TCP сервер.

Листинг 3.6 — Код работы тестового соединения, сервер

```
import socket

HOST = "127.0.0.1"  # Standard loopback interface address (localhost)
PORT = 65432  # Port to listen on (non-privileged ports are > 1023)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    conn, addr = s.accept()
    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024)
            print(str(data, 'utf-8'))
            res = b'OK'
            conn.sendall(res)
```

Создаем TCP клиент:

Листинг 3.7 — Код работы тестового соединения, клиент

```
import socket

HOST = "127.0.0.1"  # The server's hostname or IP address
PORT = 65432  # The port used by the server

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b"Hello, world")
    data = s.recv(1024)

print(f"Received {data!r}")
```

Результат работы представлен на Рисунках 3.11-3.12.

```
PS D:\Захар\диплом код\Diplom\tg\tcp debug> python '.\tcp client debug.py'
Received b'OK'
```

Рисунок 3.11 — Запуск тестового TCP клиента

```
PS D:\Захар\диплом код\Diplom\tg\tcp debug> python '.\tcp server debug.py'
Connected by ('127.0.0.1', 62576)
Hello, world
```

Рисунок 3.12 — Запуск тестового TCP сервера

В текущей системе и клиент и сервер находятся в одной локальной сети, IP адрес сервера является статическим, что позволит довольно просто производить подключение к стендовому оборудованию.

3.4.3 Создание панели конфигурации

Панель конфигурации, а именно ее бекенд часть, состоит из различных HTTP маршрутов, позволяющих изменять то, что находится в базе данных, откуда данные о конфигурации берет бот.

Были установлены необходимые библиотеки и инициализирован проект. Также было создано виртуальное окружение для разграничения пакетов и удобства использования. Пример кода создания админ-панели представлен на Листинге 3.8.

Листинг 3.8 — Часть кода работы админ панели

```
from fastapi import FastAPI
from db import get_configs, create_config
from models import Config

app = FastAPI()


@app.get("/get_configs")
async def root():
    res = get_configs()
    return res


@app.post("/add_config")
async def root(config: Config):
    res = create_config(config)
    return res
```

В коде есть несколько основополагающих модулей. В них входит модуль-обработчик HTTP запросов и модуль взаимодействия с БД.

Полный код админ панели представлен в Приложении Г.

Запустим панель конфигурации. Запуск и пример интерфейса Swagger представлены на Рисунке 3.13-3.14 соответственно.

```
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\Захар\диплом код\Diplom\panel> uvicorn main:app
INFO:     Started server process [16024]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Рисунок 3.13 — Запуск тестового сервера FastAPI

FastAPI предоставляет Swagger интерфейс, благодаря чему можно легко отлаживать проект и смотреть результаты HTTP запросов.

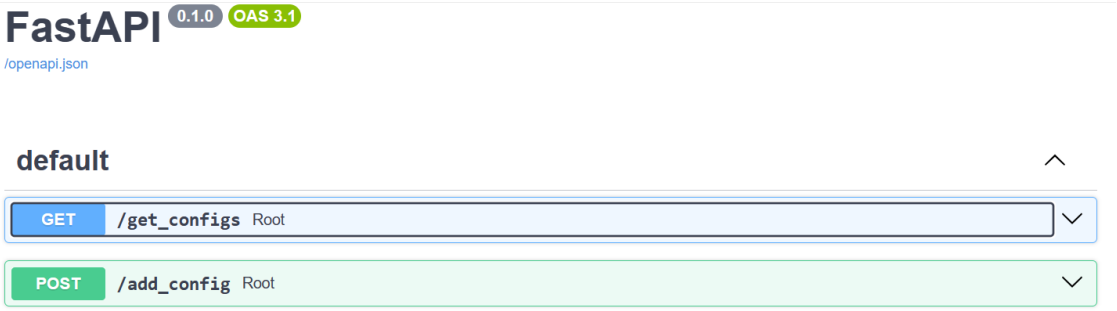

Рисунок 3.14 — Маршруты FastAPI

При создании конфигурации данные записываются в БД, при получении данных, возвращается JSON файл, содержащий информацию обо всех конфигурациях. На Рисунке 3.15 представлен пример тела HTTP запроса в Swagger.

Рисунок 3.15 — Результат выполнения GET запроса

Данный способ взаимодействия позволит легко изменять данные в БД, не зависимо от фронтенда. На текущий момент представлены не все типы запросов, которые будут приниматься на сервере, также, помимо GET и POST запросов, будут использоваться запросы PUT и DELETE запросы для обновления и удаления данных.

3.4.4 Интеграция голосового помощника в систему

Для интеграции голосового помощника необходимо воспользоваться инструментом speech-to-text.

В языке программирования Python существует библиотека speech_recognition, предоставляющая доступ к API Google, что сделает возможным транслировать голосовые сообщения в текст.

Необходимо добавить в код бота обработчик голосовых сообщений, который будет переводить сообщения в текст, после чего обрабатывать по той же логике, что и другие команды. Пример кода представлен в Листинге 3.9

Листинг 3.9 — Часть кода голосового помощника

```
import speech_recognition as sr
import soundfile


def recognize():
    r = sr.Recognizer()
    data, samplerate = soundfile.read('tmp.wav')
    soundfile.write('tmp.wav', data, samplerate, subtype='PCM_16')
    harvard = sr.AudioFile('tmp.wav')
    with harvard as source:
        audio = r.record(source)

    text = r.recognize_google(audio, language='ru-RU')
    return text
```

Далее в тексте проверяем наличие определенны ключевых слов, таких как «кнопка», «один» и «включить», и отправлять необходимый запрос в случае наличия такового.

3.4.5 Создание дашборда для интерактивного представления информации

Для создания дашборда будет использована библиотека Dash, позволяющая создавать и поддерживать дашборды в реальном времени.

Для этого понадобится воспользоваться библиотекой pandas для чтения данных из БД и представления их в удобном формате. После чего будет произведена визуализация данных через библиотеку matplotlib, и полученные графики будут встроены в html документ. Пример графиков, используемых в дашборде демонстрирован на Рисунке 3.16-3.17.

Выбранные для визуализации графики представляют собой 2 диаграммы:

- Pie Plot (круговая диаграмма) для визуализации распределения различных типов действий пользователей, чтобы показать, какие действия они выполняли чаще;

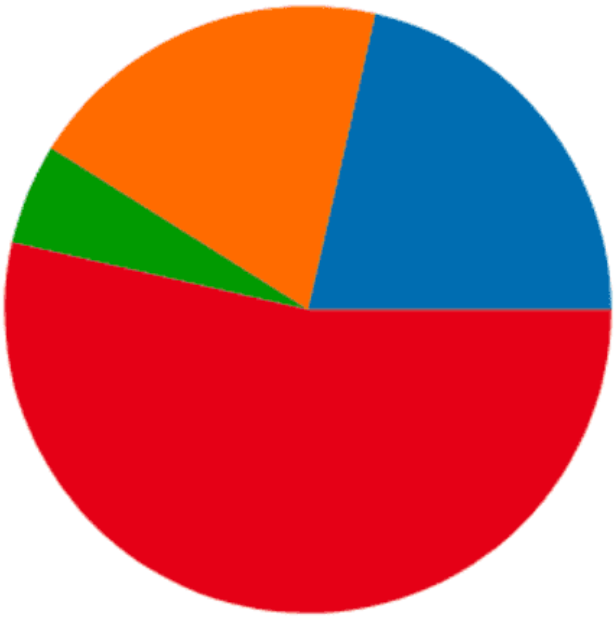

Рисунок 3.16 — Pie Plot

- Line Plot (линейная диаграмма) для визуализации изменения количества запросов определенного типа во времени;

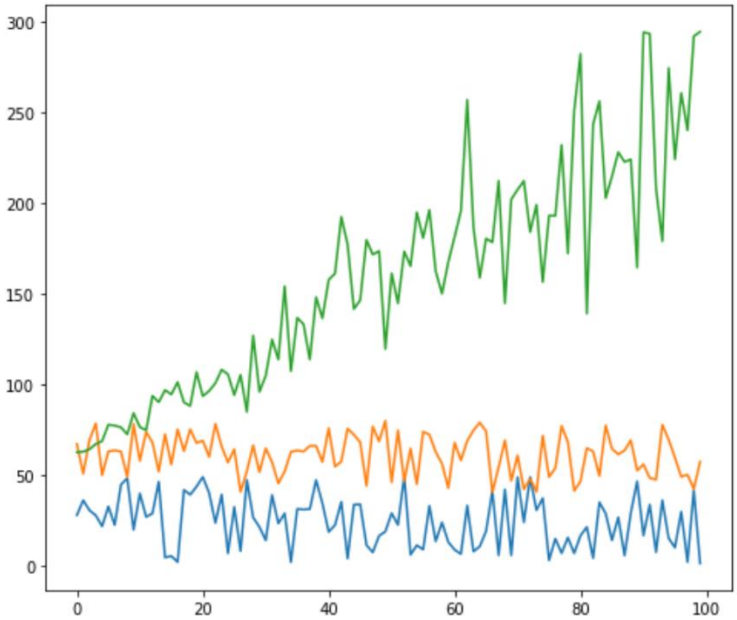

Рисунок 3.17 — Line Plot

Для создания дашборда был написан код, который получает уже агрегированные данные из view базы данных, переводит их в pandas, визуализирует через matplotlib, встраивает их в html и запускает сервер на Flask.

В Листинге представлена часть кода, отвечающая за создание дашборда.

Листинг 3.10 — Часть кода создания дашборда

```
import dash
import dash_core_components as dcc
import dash_html_components as html
from dash.dependencies import Input, Output
import pandas as pd
from db import read_distibution, read_dynamic
import plotly.express as px

dfs = read_dynamic()
df = read_distibution()

app = dash.Dash(__name__)

timeline_1 = px.line(dfs[0].sort_values('hour'), x="hour", y="cnt",
title='Динамика действий бота, двигатель')
timeline_2 = px.line(dfs[1].sort_values('hour'), x="hour", y="cnt",
title='Динамика действий бота, лампочки')
pie = px.pie(df, values='cnt', names='action_name', title='Распределение
действий бота')

app.layout = html.Div([
    html.H1("Действия бота"),
    dcc.Graph(figure=timeline_1),
    dcc.Graph(figure=timeline_2),
    dcc.Graph(figure=pie)
])

if __name__ == '__main__':
    app.run_server(debug=True)
```

Весь код создания дашборда представлен в Приложении Д.

Итоговый дашборд представлен на Рисунках 3.18-3.19.

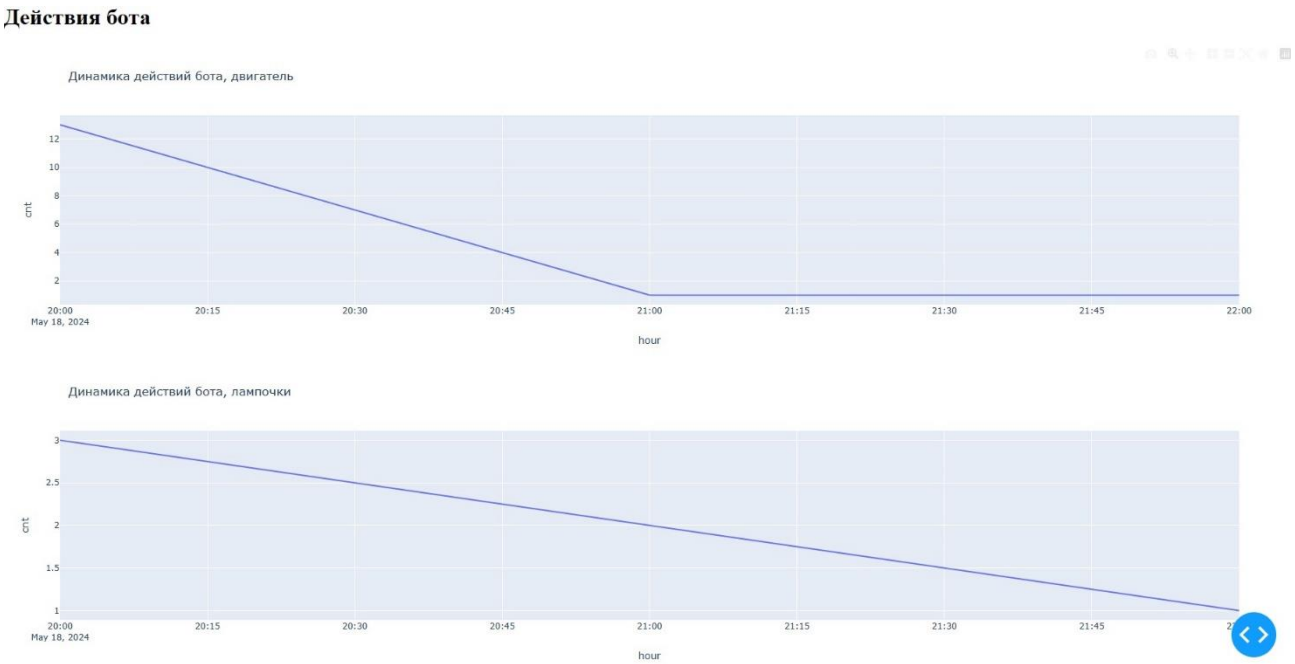

Рисунок 3.18 — Первая страница дашборда

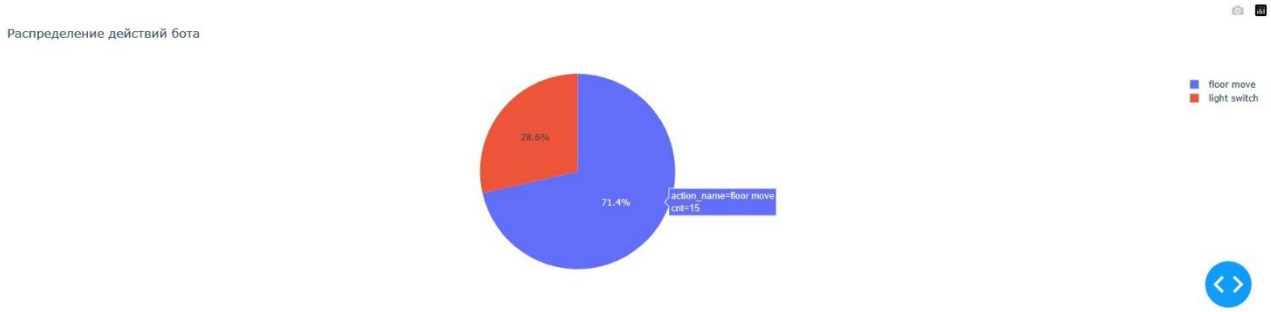

Рисунок 3.19 — Вторая страница дашборда

3.4.6 Сборка и развертка проекта

На последнем этапе создания проекта были созданы Dockerfiles для каждого отдельного приложения, написанного на языке Python.

Это поможет создать образ из созданного ранее приложения, так как воспользоваться готовым образом в данной ситуации не получится.

Dockerfile для проектов на Python используется для создания образа Docker, который содержит все необходимые зависимости и настройки для запуска приложения. Вот как работает Dockerfile для таких проектов:

- определение базового образа. В начале Dockerfile указывается базовый образ, например, FROM python:3.11, где python:3.11 — это официальный образ Python версии 3.11. Это базовый образ, на котором будет строиться ваше приложение;

- установка зависимостей: Зависимости приложения, указанные в файле requirements.txt, устанавливаются с помощью команды RUN pip install –no-cache-dir -r requirements.txt. Это гарантирует, что все необходимые библиотеки будут установлены в образе;
- копирование исходного кода приложения: Исходный код приложения копируется из локальной директории в образ с помощью команды COPY. /app — это рабочая директория внутри контейнера, куда будет скопирован код;
- указание команды запуска. Команда, которая запускает приложение, указывается с помощью директивы CMD. CMD [«python», «app.py»] запустит файл app.py;
- открытие портов. Если приложение слушает определенный порт, этот порт должен быть открыт в Dockerfile с помощью директивы EXPOSE.

Пример кода создания конфигурации в Docker представлен в Листинге 3.11

Листинг 3.11 — Деплой сервисов

```
version: '3'
services:
  api:
    build: ./panel
    ports:
      - "8080:8080"
  dashboard:
    build: ./dashboard
    ports:
      - «8050:8050»
  bot:
    build: ./tg
```

В docker-compose представлена конфигурация, предназначенная для развертки приложения. Группа состоит из 3 контейнеров:

- контейнер бота (bot);
- контейнер панели конфигурации (api);
- контейнер дашборда (dashboard).

База данных работает автономно вне этой группы контейнеров, она является автономной.

Пример Dockerfile для бота представлен в Листинге 3.12.

Листинг 3.12 — Деплой Telegram бота

```
FROM python:3.11

WORKDIR /app

COPY ./requirements.txt .

RUN pip install -no-cache-dir -r requirements.txt

COPY . .

CMD [“python”, “bot.py”]
```

На Рисунке 3.20 видно, что все контейнеры работают исправно

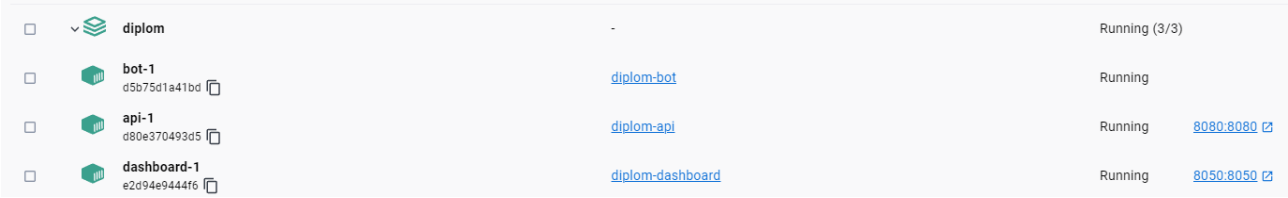

Рисунок 3.20 — Контейнеры приложений

Вся конфигурация Docker представлена в Приложении Е.

4 ЭКОНОМИЧЕСКИЙ РАЗДЕЛ

4.1 Организация и планирование работ по теме

Организация работ по теме «Информационно-управляющая система интерактивного взаимодействия со стендовым оборудованием» осуществляется на базе календарного планирования, которое включает в себя:

- определение участников проекта;
- планирование этапов проекта и декомпозицию работ;
- определение последовательности работ;
- планирование сроков, длительностей и логических связей работ;
- построение диаграммы Ганта.

В проекте принимают участие 4 человека.

Руководитель (Курнасов Е.В., к.т.н., доцент кафедры ПИ РТУ МИРЭА) — организует, планирует, оценивает и корректирует деятельность разработчиков, оценивает получаемые результаты, а также контролирует отдельные этапы работ, вносит необходимые коррективы и оценивает выполненный проект в целом.

Консультант (Казанская И.В., к.э.н., доцент кафедры экономики РТУ МИРЭА) — контролирует организацию и планирование всех работ по теме, оценивает правильность расчёта стоимости выполненных работ.

Консультант (Антонов С.В., старший преподаватель кафедры ПИ РТУ МИРЭА) — отвечает за консультирование по разработке ИУС интерактивного взаимодействия со стендовым оборудованием.

Разработчик (Иванисенко З. И., студент группы ИВБО-07-20) — реализует все этапы ВКР, в том числе проектирование ИУС и программную реализацию.

Состав этапов создания ИУС интерактивного взаимодействия со стендовым оборудованием и входящих в них работ представлен в Таблице 4.1.

Таблица 4.1 — Календарный план выполнения проекта

Этап	Дата начала	Дата окончания	Кол-во рабочих дней	Исполнители
1. Формирование требований к проекту				
1.1. Исследование существующих систем интерактивного взаимодействия со стендовым оборудованием	09.02.2024	15.02.2024	6	Разработчик проекта
1.2 Формирование требований пользователя к проекту	16.02.2024	17.02.2024	2	Руководитель ВКР Разработчик проекта
1.3 Оформление отчета о выполненной работе	19.02.2024	19.02.2024	1	Разработчик проекта
2. Разработка концепции проекта				
2.1 Изучение объекта исследования	20.02.2024	20.02.2024	1	Разработчик проекта
2.2 Проведение необходимых исследовательских работ	21.02.2024	21.02.2024	1	Разработчик проекта
2.3 Разработка вариантов взаимодействия со стендовым оборудованием	22.02.2024	26.02.2024	3	Руководитель ВКР Разработчик проекта Консультант по специальной части
2.4 Оценка рисков проекта	27.02.2024	27.02.2024	1	Разработчик проекта
2.5 Оформление отчета о выполненной работе	28.02.2024	01.03.2024	3	Разработчик проекта
3. Техническое задание				
3.1 Разработка и утверждение технического задания на разработку ИУС	02.03.2024	02.03.2024	1	Разработчик проекта
3.2. Разработка документации	04.03.2024	07.03.2024	4	Разработчик проекта
4. Эскизный (пилотный) проект				
4.1 Проектировка систем по взаимодействию со стендовым оборудованием	09.03.2024	23.03.2024	13	Разработчик проекта
4.2 Разработка систем для взаимодействия со стендовым оборудованием	25.03.2024	29.03.2024	5	Разработчик проекта

Продолжение Таблицы 4.1

Этап	Дата начала	Дата окончания	Кол-во рабочих дней	Исполнители
4.3 Создание документации для работы с стендовым оборудованием	30.03.2024	01.04.2024	2	Разработчик проекта
5. Технический проект				
5.1 Разработка итоговых проектных решений	02.04.2024	06.04.2024	5	Разработчик проекта
5.2 Разработка итогового решения взаимодействия со стендовым оборудованием	08.04.2024	12.04.2024	5	Разработчик проекта
5.3 Доработка решения взаимодействия со стендовым оборудованием	13.04.2024	18.04.2024	5	Разработчик проекта
5.4 Разработка документации на ИУС интерактивного взаимодействия со стендовым оборудованием	19.04.2024	20.04.2024	2	Разработчик проекта
6. Рабочая документация				
6.1 Разработка рабочей документации на ИУС интерактивного взаимодействия со стендовым оборудованием	22.04.2024	27.04.2024	6	Разработчик проекта
6.2 Формирование комплекта рабочей документации на ИУС интерактивного взаимодействия со стендовым оборудованием	29.04.2024	30.04.2024	2	Разработчик проекта
6.3 Расчёт затрат на проведение работ	02.05.2024	02.05.2024	1	Разработчик проекта Консультант по экономической части
7. Ввод в действие				
7.1 Подготовка объекта автоматизации к вводу решения взаимодействия со стендовым оборудованием в действие	03.05.2024	04.05.2024	2	Разработчик проекта

Окончание Таблицы 4.1

Этап	Дата начала	Дата окончания	Кол-во рабочих дней	Исполнители
7.2 Обучение персонала	06.05.2024	06.05.2024	1	Разработчик проекта
7.3 Комплектация поставляемыми изделиями (программными и техническими средствами, программно-техническими комплексами, информационными изделиями)	07.05.2024	07.05.2024	1	Разработчик проекта
7.4 Тестирование решения взаимодействия со стендовым оборудованием	08.05.2024	10.05.2024	2	Разработчик проекта
7.5 Ввод решения взаимодействия со стендовым оборудованием в эксплуатацию	11.05.2024	14.05.2024	3	Разработчик проекта
7.6 Проведение приемочных испытаний	15.05.2024	16.05.2024	2	Руководитель ВКР Разработчик проекта
7.7 Сопровождение апробации решения взаимодействия со стендовым оборудованием	17.05.2024	20.05.2024	3	Разработчик проекта
ИТОГО			83	

Таким образом, на разработку отводится 83 рабочих дня.

Для визуализации хода выполнения проекта, планирования ресурсов, графика рабочего времени и других данных использована диаграмма Ганта.

Календарный план-график (диаграмма Ганта) разработан в соответствии с данными Таблицы 4.1 и представлен на Рисунке 4.1

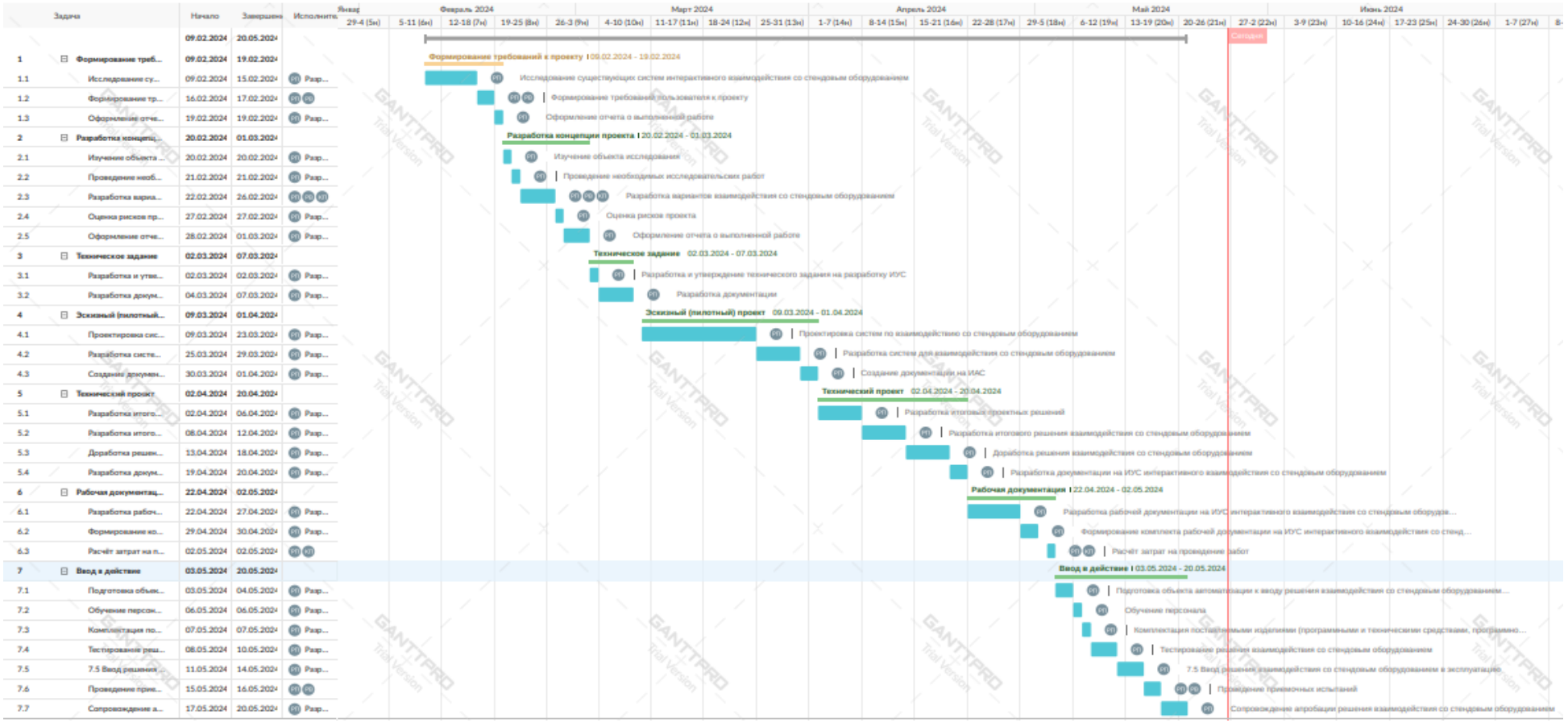

Рисунок 4.1 — Календарный план-график

4.2 Расчет затрат на проведение работ по теме

Объем финансирования разработки ИУС интерактивного взаимодействия со стендовым оборудованием складывается из затрат по 7 статьям.

Статья «Основная заработная плата» включает оплату труда разработчиков проекта. Ее расчет приведен в Таблице 4.2.

Таблица 4.2 — Расчет основной заработной платы

№ п/п	Наименование этапа	Исполнитель (должность)	Мес. Оклад (руб.)	Оплата за день (руб.)	Количество рабочих дней	Оплата за этап (руб.)
1	Формирование требований к проекту	Руководитель ВКР	105000	4200	2	8400
		Разработчик проекта	0	0	9	0
2	Разработка концепции проекта	Руководитель ВКР	105000	4200	3	12600
		Разработчик проекта	0	0	9	0
		Консультант по специальной части	90000	3600	3	10800
3	Техническое задание	Разработчик проекта	0	0	5	0
4	Эскизный (пилотный) проект	Разработчик проекта	0	0	20	0
5	Технический проект	Разработчик проекта	0	0	17	0
6	Рабочая документация	Разработчик проекта	0	0	9	0
		Консультант по экономической части	105000	4200	1	4200
7	Ввод в действие	Руководитель ВКР	105000	4200	2	8400
		Разработчик проекта	0	0	14	0
Всего занятость		Руководитель ВКР			7	29400
		Консультант по специальной части			3	10800
		Консультант по экономической части			1	4200
		Разработчик проекта			83	0
ИТОГО основная заработная плата исполнителей						44400

Годовой фонд рабочего времени при шестидневной рабочей неделе принимаем 300 рабочих дней. Дневная оплата (ТС) для месячного оклада (ОК) по каждому участнику проекта рассчитана следующим образом (Формула 4.1):

$$\text{ТС} = \frac{\text{ОК}\times 12}{\text{НРВ}} = \frac{OK\times 12}{300}\,\text{руб./день} \qquad (4.1)$$

Статья «Дополнительная заработная плата» включает выплаты, предусмотренные законодательством о труде: за неотработанное по уважительным причинам время; оплата очередных и дополнительных отпусков; оплата времени, связанного с выполнением государственных и общественных обязанностей и т.п. Расходы по данной статье принимаем в размере 20% от основной заработной платы:

$$\text{ДЗП} = 0{,}2 \cdot 44400 = 8880\ (\text{руб.})$$

Фонд оплаты труда разработчиков проекта представляет собой сумму основной и дополнительной заработной платы:

$$\text{ФОТ} = 44400 + 8880 = 53280\ (\text{руб.})$$

Статья «Страховые взносы» включает взносы на обязательное пенсионное и медицинское страхование, а также социальное страхование на случай временной нетрудоспособности и в связи с материнством. В 2024 году единый тариф страховых взносов по этим направлениям составляет 30% от фонда оплаты труда. Кроме того, в расчете учитываем страховые взносы на травматизм по ставке 0,2%. Таким образом:

$$\text{СВ} = 0{,}302 \cdot 53280 = 16090\ (\text{руб.})$$

Статья «Материалы, покупные изделия и полуфабрикаты» учитывает стоимость материалов, покупных изделий, полуфабрикатов, комплектующих изделий и других материальных ценностей, расходуемых непосредственно в процессе разработки системы интерактивного взаимодействия со стендовым оборудованием.

Затраты на канцелярские принадлежности рассчитаны в Таблице 4.3.

Таблица 4.3 — Расчет стоимости материалов

№ п/п	Наименование материалов	Единицы измерения	Количество	Цена за единицу (руб.)	Стоимость (руб.)
1	USB-флеш-накопитель	шт.	1	600	600
2	Бумага А 4	пачка	1	400	400
3	Картридждля принтера	шт.	1	2000	2000
4	Канцелярский набор	шт.	1	900	900
5	Кабель патч-корд	шт.	10	150	1500
Итого материалов					5 400
Транспортно-заготовительные расходы (10%)					540
ИТОГО:					5 940

Статья «Расходы на приобретение компьютерной техники и программного обеспечения». Расчет стоимости используемого компьютерного оборудования приведен в Таблице 4.4.

Таблица 4.4 — Расчет стоимости компьютерного оборудования

№ п/п	Наименование оборудования	Количество	Цена за единицу (руб.)	Стоимость (руб.)
1	Монитор	4	11000	44000
2	Процессор	2	16000	32000
3	Материнская плата	2	10000	20000
4	Оперативная память	4	4000	16000
5	Видеокарта	2	18000	36000
6	Блок питания	2	4000	8000
7	Кулер	6	1500	9000
8	Корпус ПК	2	4000	8000
9	Компьютерная мышь	2	2000	4000
10	Клавиатура	2	3000	6000
11	Принтер	1	12000	12000
Итого оборудования				195000
Транспортно-заготовительные расходы (20%)				197500
ИТОГО:				392500

Расчет стоимости используемого программного обеспечения приведен в Таблице 4.5.

Таблица 4.5 — Расчет стоимости программного обеспечения

№ п/п	Наименование программного обеспечения	Цена за единицу (руб.)
1	Операционная система «Windows 10»	2500
2	Erwin data modeler	0
3	PyCharm	0
4	PostgreSQL	0
ИТОГО:		2500

Поскольку оборудование участниками проекта было предоставлено на безвозмездной основе, то расходы по статье «Расходы на приобретение компьютерной техники и программного обеспечения» отсутствуют.

Статья «Эксплуатационные расходы, связанные с использованием компьютерной техники».

Под эксплуатационными расходами понимаются затраты на электроэнергию, используемую при работе оборудования. При расчетах исключены затраты для комплектующих изделий ввиду их малого энергопотребления. В Таблице 4.6 приведен их расчет.

Таблица 4.6 — Расчет эксплуатационных расходов

Наименование техники	Мощность, кВт	Время использования, ч.	Стоимость 1 кВт·ч, руб.	Расходы, руб.
ПК для разработки	0,35	664	6.43	1656
ПК для аналитики	0,7	344		1548
Принтер	0,02	2		1
Мониторы для разработки	0,72	368		1704
Мониторы для аналитики	0,72	344		1593
ИТОГО:				6500

Статья «Накладные расходы». В рамках реализации проекта это расходы, связанные с содержанием и арендой площадей, с приобретением расходных материалов для офиса (не учтенных в статье «Материалы, покупные изделия и полуфабрикаты»), транспортными расходами, оплатой консультационных, информационных услуг, а также с командировочными расходами.

Сумма накладных расходов определяется в размере 100% от основной заработной платы участников проекта:

НР = 100% от 44400 = 44400 (руб.)

Вышеперечисленные статьи затрат и результаты расчётов по ним сведены в Таблицу 4.7.

Таблица 4.7 — Объем финансирования проекта

П № п/п	Номенклатура статей расходов	Расходы (руб.)	Доля расходов, %
1	Основная заработная плата	44400	35.2
2	Дополнительная заработная плата	8880	7
3	Страховые взносы	16090	12.7
4	Материалы, покупные изделия и полуфабрикаты	5940	4.7
5	Расходы на приобретение компьютерной техники и программного обеспечения	0	0
6	Эксплуатационные расходы, связанные с использованием компьютерной техники	6500	5.2
7	Накладные расходы	44400	35.2
Итого:		126210	100

Для визуализации структуры затрат в общем объеме финансирования проекта построена круговая диаграмма (Рисунок 4.2).

Рисунок 4.2 — Структура затрат по проекту

Рассчитаем договорную цену ИУС интерактивного взаимодействия со стендовым оборудованием (Формула 4.3):

$$Ц = С + П + НДС, \quad (4.3)$$

где С – стоимость проекта, рассчитанная в Таблице 4.8;

П – прибыль (принимаем 30% от стоимости проекта);

НДС – налог на добавленную стоимость (20%).

П = 0,3 · 126210= 37860 (руб.)

НДС = 0,2 · (С + П) = 0,2 · (126210 + 37860) = 32810 (руб.)

Таким образом, договорная цена:

Ц = 126210 + 37863 + 32810 =196900 (руб.).

ЗАКЛЮЧЕНИЕ

Проанализировав предметную область, были определены ключевые требования к ИУС. Структурно-функциональный анализ в нотациях IDEF0 и BPMN позволил четко представить систему и процессы. Разработка архитектуры ИУС обеспечила гибкость и масштабируемость системы. Логическая модель данных обеспечила единообразие и согласованность данных. На основе спроектированной системы были реализованы различные ее компоненты. В результате, была создана эффективная информационная система, отвечающая требованиям и ожиданиям пользователей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Полное руководство по умному дому от MiHome — URL: https://mi92.ru/blog-xiaomi-na-russkom/stati/polnoe-rukovodstvo-po-umnomu-domu-xiaomi (дата обращения: 09.03.2024).

2. Умный дом с Алисой от Яндекса. — URL: https://habr.com/ru/companies/67andex/articles/453080/ (дата обращения: 09.03.2024).

3. Протоколы TCP и ModbusTCP — URL: https://www.cta.ru/articles/cta/spravochnik/v-zapisnuyu-knizhku-inzhenera/124866/ (дата обращения: 11.03.2024).

4. Что такое протокол HTTP и почему на нём работают почти все сайты — URL: https://practicum.yandex.ru/blog/chto-takoe-protokol-http/ (дата обращения: 11.03.2024).

5. Правила оформления практики, курсовой работы (проекта), ВКР бакалавра/магистра — URL: https://www.swrit.ru/doc/gost34/34.602-2020.pdf (дата обращения: 12.03.2024).

6. API (application programming interface) – общие сведения – URL: https://www.unisender.com/ru/glossary/chto-takoe-api/#anchor-1 (дата обращения: 04.04.2024).

7. Основы правил проектирования базы данных – URL: https://tlgrm.ru/docs/bots (дата обращения: 03.04.2024).

8. Боты: информация для разработчиков – URL: https://habr.com/ru/articles/514364/ (дата обращения: 04.04.2024).

9. Руководство по использованию BPMN в повседневной работе – URL: https://www.microsoft.com/ru-ru/microsoft-365/business-insights-ideas/resources/the-guide-to-using-bpmn-in-your-business (дата обращения: 04.04.2024).

10. Диаграмма Ганта [Электронный ресурс]. — 2023. URL: https://v8.1c.ru/platforma/diagramma-ganta/ (дата обращения: 11.04.2024).

11. GanttPRO: Диаграмма Ганта для управления проектами [Электронный ресурс]. — 2023. URL: https://ganttpro.com/ru/ (дата обращения: 11.04.2024).

12. Единый тариф (единые размеры) страховых взносов на ОПС, ОСС, ОМС для плательщиков, производящих выплаты физическим лицам, на 2024 г. [Электронный ресурс]. — 2024. URL: https://www.consultant.ru/document/cons_doc_LAW_93256/977315fdb3b184feec5e6517d4866e602980ba76/ (дата обращения: 12.04.2024).

13. Федеральный закон от 24.07.1998 N 125-ФЗ (ред. От 25.12.2023) «Об обязательном социальном страховании от несчастных случаев на производстве и профессиональных заболеваний» (с изм. И доп., вступ. В силу с 01.01.2024) [Электронный ресурс]. — 2024. URL: https://www.consultant.ru/document/cons_doc_LAW_19559/ (дата обращения: 15.04.2024).

14. Федеральный закон от 27.11.2023 N 549-ФЗ «О страховых тарифах на обязательное социальное страхование от несчастных случаев на производстве и профессиональных заболеваний на 2024 год и на плановый период 2025 и 2026 годов». [Электронный ресурс]. — 2024. URL: https://www.consultant.ru/document/cons_doc_LAW_462887/ (дата обращения: 15.04.2024).

15. Методические рекомендации по выполнению организационно-экономической части выпускных квалификационных работ [Электронный ресурс]: метод. Указания / Т. Ю. Гавриленко, О. В. Григоренко, Е. К. Ткаченко. — М.: РТУ МИРЭА, 2019. — Электрон. Опт. Диск (ISO). (дата обращения: 16.04.2024).

16. Экономика предприятия [Электронный ресурс]: учебное пособие / А. С. Вихрова. — М.: РТУ МИРЭА, 2021. — Электрон. Опт. Диск (ISO). (дата обращения: 16.04.2024).

ПРИЛОЖЕНИЯ

Приложение А — Создание базы данных.

Приложение Б — Разработка серверного компонента системы.

Приложение В — Реализация Telegram бота.

Приложение Г — Реализация панели конфигурации на стороне сервера.

Приложение Д — Создание дашборда.

Приложение Е — Деплой проекта.

Приложение А

Листинг А.1 — Скрипт создания БД

```
DROP TABLE IF EXISTS Scenario;
CREATE TABLE Scenario (
    scen_id SERIAL,
    scen_desc TEXT,
    scen_name TEXT,
    PRIMARY KEY (scen_id)
);

DROP TABLE IF EXISTS Button;
CREATE TABLE Button (
    button_id SERIAL,
    button_name TEXT,
    scen_id INTEGER,
    PRIMARY KEY (button_id),
    FOREIGN KEY (scen_id) REFERENCES Scenario(scen_id)
);

DROP TABLE IF EXISTS Var;
CREATE TABLE Var (
    variable_id SERIAL,
    var_name TEXT,
    var_type TEXT,
    PRIMARY KEY (variable_id)
);

DROP TABLE IF EXISTS Config;
CREATE TABLE Config (
    variable_id SERIAL,
    button_id INTEGER NOT NULL,
    config_id INTEGER NOT NULL,
    PRIMARY KEY (config_id),
    FOREIGN KEY (variable_id) REFERENCES Var(variable_id),
    FOREIGN KEY (button_id) REFERENCES Button(button_id)
);

DROP TABLE IF EXISTS Action;
CREATE TABLE Action (
    action_id SERIAL,
    action_name TEXT,
    action_desc TEXT,
    PRIMARY KEY (action_id)
);

DROP TABLE IF EXISTS Analytics;
CREATE TABLE Analytics (
    log_id SERIAL,
    insert_time TIMESTAMP,
    action_id INTEGER,
    PRIMARY KEY (log_id),
    FOREIGN KEY (action_id) REFERENCES Action(action_id)
);

INSERT INTO Action VALUES
(1, ‘floor move’, ‘Move up to a specific floor’),
(2, ‘light switch’, ‘Switch the light on a button’);

DROP VIEW IF EXISTS analytics_log;
CREATE VIEW analytics_log AS
SELECT b.action_name, DATE_TRUNC(‘hour’, insert_time) AS hour, COUNT(*) AS cnt
```

Продолжение Листинга А.1

```
FROM Analytics AS a
INNER JOIN Action AS b
ON a.action_id = b.action_id
GROUP BY b.action_name, DATE_TRUNC('hour', insert_time)
ORDER BY hour;

DROP VIEW IF EXISTS analytics_log_distr;
CREATE VIEW analytics_log_distr AS
SELECT b.action_name, COUNT(*) AS cnt
FROM Analytics AS a
INNER JOIN Action AS b
ON a.action_id = b.action_id
GROUP BY b.action_name;
```

Приложение Б

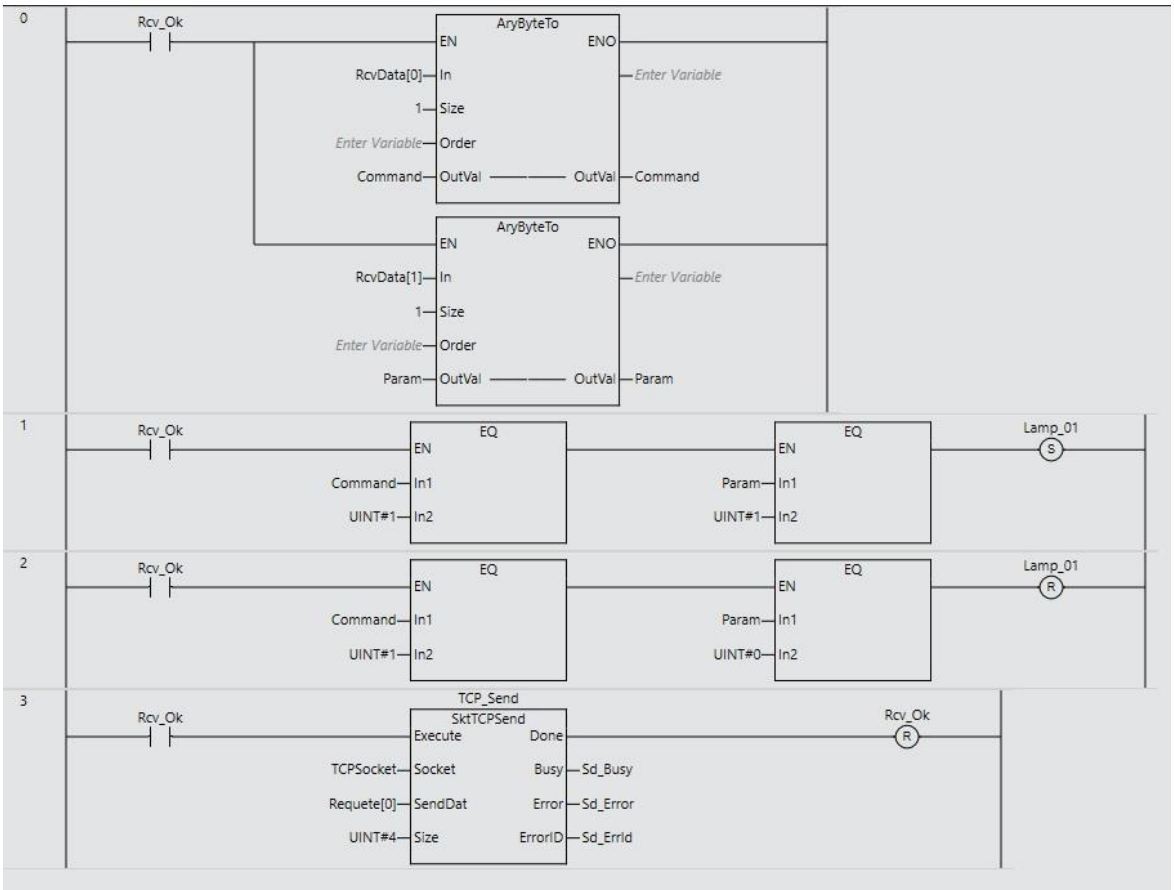

Рисунок Б.1 — Скрипт парсинга данных в переменные

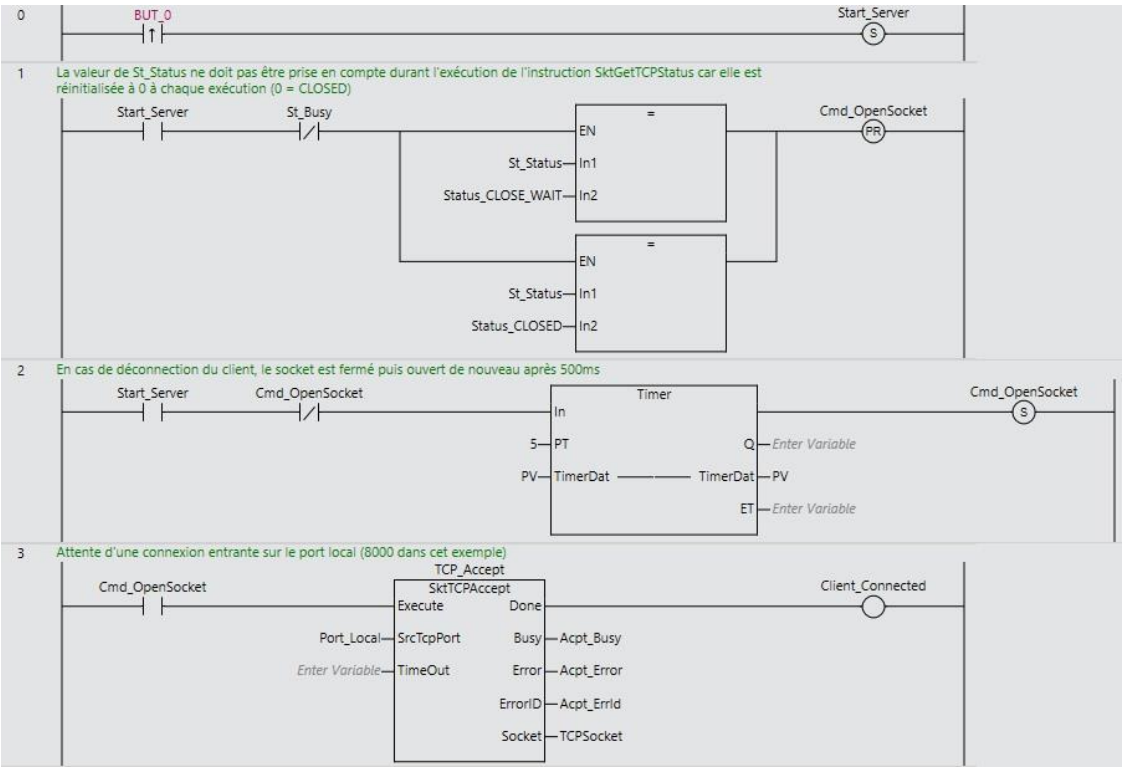

Рисунок Б.2 — Скрипт TCP соединения

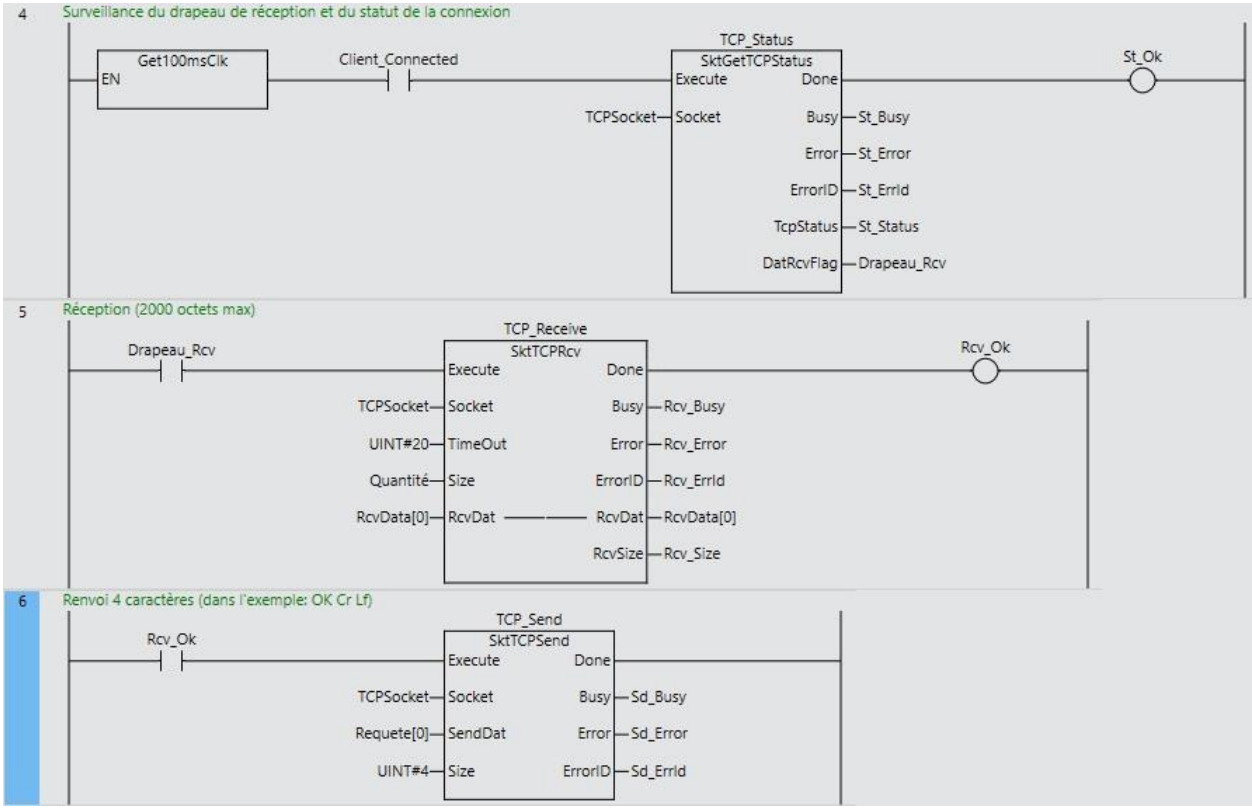

Рисунок Б.3 — Скрипт TCP соединения

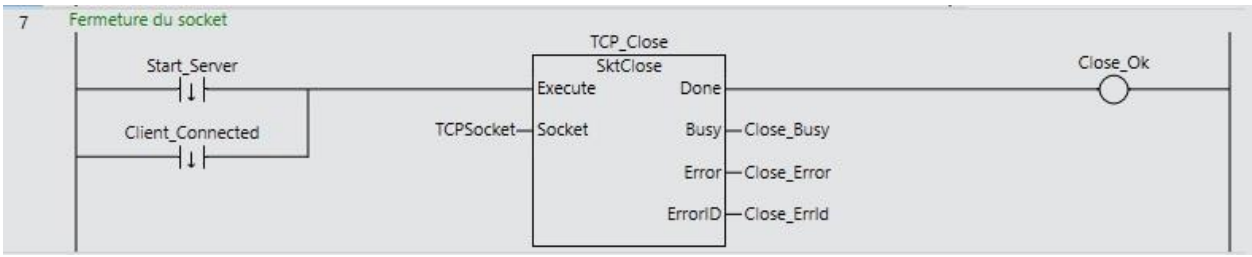

Рисунок Б.4 — Скрипт TCP соединения

Приложение В

Листинг В.1 — Создание Telegram бота

```
import telebot
from telebot import types

from config import TOKEN
from db import insert_action_data, get_command
from TCP import send_TCP_request
from speech_encoder import recognize

bot = telebot.TeleBot(TOKEN, parse_mode=None)


def act(button_name, param, call, action_type_id):
    insert_action_data(action_type_id)
    command = get_command(button_name, param)
    res = send_TCP_request(f'{command[0]}{command[1]}')
    bot.send_message(call.message.chat.id, res)


def voice_act(button_name, param, message, action_type_id):
    insert_action_data(action_type_id)
    command = get_command(button_name, param)
    res = send_TCP_request(f'{command[0]}{command[1]}')
    bot.send_message(message.chat.id, res)


@bot.message_handler(content_types=['voice'])
def process_audio(message):
    file_info = bot.get_file(message.voice.file_id)
    downloaded_file = bot.download_file(file_info.file_path)
    with open('tmp.wav', 'wb') as new_file:
        new_file.write(downloaded_file)
    text = recognize()
    if 'этаж' in text:
        if '2' in text:
            voice_act('button_2', '02', message, 1)


@bot.message_handler(commands=['move'])
def move(message):
    keyboard = types.InlineKeyboardMarkup(row_width=2)  # Создаем клавиатуру с
двумя кнопками в строке
    btn_1 = types.InlineKeyboardButton("Этаж 1", callback_data='button_1')
    btn_2 = types.InlineKeyboardButton("Этаж 2", callback_data='button_2')
    btn_3 = types.InlineKeyboardButton("Этаж 3", callback_data='test')
    keyboard.add(btn_1, btn_2, btn_3)  # Добавляем кнопки на клавиатуру
    bot.send_message(message.chat.id, 'Выберите кнопку:', reply_markup=keyboard)
# Отправляем сообщение с клавиатурой


@bot.message_handler(commands=['light'])
def light(message):
    keyboard = types.InlineKeyboardMarkup(row_width=2)  # Создаем клавиатуру с
двумя кнопками в строке
    btn_1 = types.InlineKeyboardButton("Лампочка 1 выкл",
callback_data='button_3')
    btn_2 = types.InlineKeyboardButton("Лампочка 1 вкл",
callback_data='button_4')
    btn_3 = types.InlineKeyboardButton("Лампочка 3", callback_data='test')
    keyboard.add(btn_1, btn_2, btn_3)  # Добавляем кнопки на клавиатуру
```

Окончание Листинга В.1

```
    bot.send_message(message.chat.id, 'Выберите кнопку:', reply_markup=keyboard)
# Отправляем сообщение с клавиатурой


@bot.callback_query_handler(func=lambda call: True)  # Обработчик для callback-
запросов
def callback_inline(call):
    insert_action_data(1)
    if call.data == 'button_1':
        act(call.data, '01', call, 1)
    elif call.data == 'button_2':
        act(call.data, '02', call, 1)
    elif call.data == 'button_3':
        act(call.data, '00', call, 2)
    elif call.data == 'button_4':
        act(call.data, '00', call, 2)
    elif call.data == 'test':
        act(call.data, '00', call, 1)


bot.infinity_polling()
```

Листинг В.2 — Создание переменных оркужения

```
from dotenv import load_dotenv
import os

load_dotenv()

TOKEN = os.getenv('TOKEN')
POSTGRES_USER = os.getenv('POSTGRES_USER')
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD')
POSTGRES_DB = os.getenv('POSTGRES_DB')
POSTGRES_HOST = os.getenv('POSTGRES_HOST')
POSTGRES_PORT = os.getenv('POSTGRES_PORT')
```

Листинг В.3 — Запросы в БД

```
import psycopg2
from db_connect import connect_db
from datetime import datetime


def get_command(button_name: str, param):
    conn = connect_db()
    cur = conn.cursor()
    select_statement = """
        SELECT var_name FROM Config
        INNER JOIN Var
        ON Var.variable_id = Config.variable_id
        INNER JOIN Button
        ON Button.button_id = Config.button_id
        WHERE button_name = (%s);
        """
    cur.execute(select_statement, (button_name, ))
    var = cur.fetchone()[0]
    conn.close()
    print(var)
    return (var, param)


def insert_action_data(action_type_id: int):
    conn = connect_db()
```

Окончание Листинга В.3

```
  cur = conn.cursor()
    select_statement = """
    SELECT MAX(log_id) FROM Analytics;
    """
    cur.execute(select_statement)
    max_id = cur.fetchone()[0]
    cur_id = max_id + 1 if max_id else 1
    insert_statement = """
    INSERT INTO Analytics VALUES (%s, %s, %s)
    """
    cur.execute(insert_statement, (cur_id, datetime.now(), action_type_id))
    conn.commit()
    conn.close()
```

Листинг В.4 — Подключение к БД

```
import psycopg2
from config import POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST,
POSTGRES_PORT


def connect_db():
    conn = psycopg2.connect(
        host=POSTGRES_HOST,
        port=POSTGRES_PORT,
        dbname=POSTGRES_DB,
        user=POSTGRES_USER,
        password=POSTGRES_PASSWORD
    )
    return conn
```

Листинг В.5 — Инструмент speech-to-text

```
import speech_recognition as sr
import soundfile


def recognize():
    r = sr.Recognizer()
    data, samplerate = soundfile.read('tmp.wav')
    soundfile.write('tmp.wav', data, samplerate, subtype='PCM_16')
    harvard = sr.AudioFile('tmp.wav')
    with harvard as source:
        audio = r.record(source)

    text = r.recognize_google(audio, language='ru-RU')
    return text
```

Листинг В.6 — Подключение к TCP серверу

```
import socket

HOST = "127.0.0.1"
PORT = 65432


def send_TCP_request(text: str):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        s.sendall(bytes(text, 'utf-8'))
        data = s.recv(1024)
        return str(data, 'utf-8')
```

Приложение Г

Листинг Г.1 — Создание переменных окружения

```
from dotenv import load_dotenv
import os

load_dotenv()

POSTGRES_USER = os.getenv('POSTGRES_USER')
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD')
POSTGRES_DB = os.getenv('POSTGRES_DB')
POSTGRES_HOST = os.getenv('POSTGRES_HOST')
POSTGRES_PORT = os.getenv('POSTGRES_PORT')
```

Листинг Г.2 — Запросы в БД

```
from db_connect import connect_db
from models import Config, Button, Scenario, Var


def get(table_name):
    conn = connect_db()
    cur = conn.cursor()
    select_statement = f"""
                SELECT * FROM {table_name};
                """
    cur.execute(select_statement)
    res = cur.fetchall()
    return res


def get_first(cur, var, table_name):
    select_statement = f"""
            SELECT MAX({var}) FROM {table_name};
            """
    cur.execute(select_statement)
    max_id = cur.fetchone()[0]
    cur_id = max_id + 1 if max_id else 1
    return cur_id


# config
def get_configs():
    return get('Config')


def create_config(config: Config):
    conn = connect_db()
    cur = conn.cursor()
    cur_id = get_first(cur, 'config_id', 'Config')
    insert_statement = """
        INSERT INTO Config (config_id, variable_id, button_id) VALUES (%s, %s,
%s);
        """
    cur.execute(insert_statement, (cur_id, config.var_id, config.button_id))
    conn.commit()
    conn.close()


# button
def create_button(button: Button):
    conn = connect_db()
```

Оконачание Листинга Г.2

```
    cur = conn.cursor()
    cur_id = get_first(cur, 'button_id', 'Button')
    insert_statement = """
        INSERT INTO Button (button_id, button_name, scen_id) VALUES (%s, %s,
%s);
        """
    cur.execute(insert_statement, (cur_id, button.button_name, button.scen_id))
    conn.commit()
    conn.close()


def get_buttons():
    return get('Button')


# scenario
def create_scen(scen: Scenario):
    conn = connect_db()
    cur = conn.cursor()
    cur_id = get_first(cur, 'scen_id', 'Scenario')
    insert_statement = """
        INSERT INTO Scenario (scen_id, scen_desc, scen_name) VALUES (%s, %s,
%s);
        """
    cur.execute(insert_statement, (cur_id, scen.scen_desc, scen.scen_name))
    conn.commit()
    conn.close()


def get_scens():
    return get('Scenario')


# variable
def create_var(var: Var):
    conn = connect_db()
    cur = conn.cursor()
    cur_id = get_first(cur, 'variable_id', 'Var')
    insert_statement = """
        INSERT INTO Var (variable_id, var_name, var_type) VALUES (%s, %s, %s);
        """
    cur.execute(insert_statement, (cur_id, var.var_name, var.var_type))
    conn.commit()
    conn.close()


def get_vars():
    return get('Var')
```

Листинг Г.3 — Подключение к БД

```
import psycopg2
from config import POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST,
POSTGRES_PORT


def connect_db():
    conn = psycopg2.connect(
        host=POSTGRES_HOST,
        port=POSTGRES_PORT,
        dbname=POSTGRES_DB,
        user=POSTGRES_USER,
        password=POSTGRES_PASSWORD
    )
    return conn
```

Листинг Г.4 — Код API

```
from fastapi import FastAPI
import db
from models import Config, Button, Scenario, Var

app = FastAPI()

@app.get("/get_scens")
async def root():
    res = db.get_scens()
    return res

@app.post("/add_scen")
async def root(scen: Scenario):
    res = db.create_scen(scen)
    return res

@app.get("/get_buttons")
async def root():
    res = db.get_buttons()
    return res

@app.post("/add_button")
async def root(button: Button):
    res = db.create_button(button)
    return res

@app.get("/get_vars")
async def root():
    res = db.get_vars()
    return res

@app.post("/add_var")
async def root(var: Var):
    res = db.create_var(var)
    return res

@app.get("/get_configs")
async def root():
    res = db.get_configs()
    return res


@app.post("/add_config")
async def root(config: Config):
    res = db.create_config(config)
    return res
```

Листинг Г.5 — Модели API

```
from pydantic import BaseModel, Field


class Config(BaseModel):
    button_id: int
    var_id: int


class Button(BaseModel):
    button_name: str
    scen_id: int


class Scenario(BaseModel):
    scen_name: str
    scen_desc: str


class Var(BaseModel):
    var_name: str = Field(default='0x00')
    var_type: str
```

Приложение Д

Листинг Д.1 — Получение и обработка данных

```
import pandas as pd
from config import POSTGRES_DB, POSTGRES_PASSWORD, POSTGRES_PORT, POSTGRES_HOST,
POSTGRES_USER

DB_URI =
f"postgresql://{POSTGRES_USER}:{POSTGRES_PASSWORD}@{POSTGRES_HOST}:{POSTGRES_POR
T}/{POSTGRES_DB}"


def read_dynamic():
    query = "SELECT * FROM analytics_log"
    df = pd.read_sql(query, DB_URI)
    df['hour'] = df['hour'].apply(format_time)
    dfs = []
    for action in df['action_name'].unique():
        dfs.append(df.loc[df['action_name'] == action, :])
    return dfs


def read_distibution():
    query = "SELECT * FROM analytics_log_distr"
    df = pd.read_sql(query, DB_URI)
    return df


def format_date(date):
    return date.strftime('%Y-%m-%d')


def format_time(date):
    return date.strftime('%Y-%m-%d %H')
```

Листинг Д.2 — Создание переменных окружения

```
from dotenv import load_dotenv
import os

load_dotenv()

POSTGRES_USER = os.getenv('POSTGRES_USER')
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD')
POSTGRES_DB = os.getenv('POSTGRES_DB')
POSTGRES_HOST = os.getenv('POSTGRES_HOST')
POSTGRES_PORT = os.getenv('POSTGRES_PORT')
```

Листинг Д.3 — Создание дашборда

```
import dash
import dash_core_components as dcc
import dash_html_components as html
from dash.dependencies import Input, Output
import pandas as pd
from db import read_distibution, read_dynamic
import plotly.express as px

dfs = read_dynamic()
df = read_distibution()

app = dash.Dash(__name__)

timeline_1 = px.line(dfs[0].sort_values('hour'), x="hour", y="cnt",
title='Динамика действий бота, двигатель')
timeline_2 = px.line(dfs[1].sort_values('hour'), x="hour", y="cnt",
title='Динамика действий бота, лампочки')
pie = px.pie(df, values='cnt', names='action_name', title='Распределение
действий бота')

app.layout = html.Div([
    html.H1("Действия бота"),
    dcc.Graph(figure=timeline_1),
    dcc.Graph(figure=timeline_2),
    dcc.Graph(figure=pie)
])

if __name__ == '__main__':
    app.run_server(debug=True)
```

Приложение Е

Листинг Е.1 — Dockerfile бота

```
FROM python:3.11

WORKDIR /app

COPY ./requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "bot.py"]
```

Листинг Е.2 — Dockerfile API

```
FROM python:3.11

WORKDIR /app

COPY ./requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Листинг Е.3 — Dockerfile дашборда

```
FROM python:3.11

WORKDIR /app

COPY ./requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "dashboard.py"]
```